MAURIZIO LUCIANI

SERVERLESS COMPUTING

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

MAURIZIO LUCIANI



SOFTWARE & WEB DEVELOPER

SERVERLESS COMPUTING

- Fondamenti di Programmazione a Oggetti
- Introduzione a .NET e C#
- Primi passi con C#
- Gestione delle Dipendenze
- Tipi di Dato e Gestione della Memoria
- Esercitazioni Pratiche
- Introduzione alle Web API e ai Microservizi
- Architetture Avanzate e Integrazione
- Integrazione con Database
- Verso il Cloud e la Containerizzazione (anticipazione)

ABOUT ME

APRA VARGROUP

its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

SUMMARY

- Programmazione Object Oriented
- Introduzione .Net
- Installazione ambiente di sviluppo
- Hello world in .Net 8
- Linguaggio C#
- Controller
- Accesso ai dati



Programmazione Object Oriented (OOP)

La programmazione orientata agli oggetti consiste nel raggruppare, all'interno di unità di codice chiamate **classi**, sia le strutture dati sia le procedure (metodi) che operano su di esse.
Una classe rappresenta quindi un modello astratto: durante l'esecuzione del programma, a partire da essa vengono creati (istanziati) gli **oggetti**, ciascuno dei quali è un'istanza concreta di quella classe.

Un linguaggio di programmazione si definisce orientato agli oggetti quando la sua sintassi nativa consente di implementare:

- **incapsulamento**;
- **ereditarietà**;
- **polimorfismo**.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Classi

Le classi definiscono dei tipi di dato e permettono la creazione degli oggetti secondo le caratteristiche definite nella classe stessa.

La classe è composta da:

- *Attributi,* ossia variabili e/o costanti che definiscono le proprietà degli oggetti istanziabili invocando la classe; i valori inizializzati degli attributi sono ottenuti attraverso il cosiddetto costruttore della classe;
- *Metodi*, ossia procedure che operano sugli attributi.

Oggetti

Un oggetto è una istanza di una classe. Esso è dotato di tutti gli attributi e i metodi definiti dalla classe.

Dal punto di vista del calcolatore, istanziare un oggetto vuol dire allocare memoria ed eventualmente inizializzarla secondo le specifiche definite dalla classe. Molti linguaggi forniscono un supporto per l'inizializzazione automatica di un oggetto, con uno o più metodi speciali, detti costruttori.

Il codice eseguibile del programma accede a tale zona di memoria sempre e solo secondo le modalità definite dalla classe.

Incapsulamento

Per alterare lo stato interno dell'oggetto, è necessario invocarne i metodi pubblici, ed è questo lo scopo principale dell'incapsulamento. Infatti, se gestito opportunamente, esso permette di vedere l'oggetto come una *black-box*, cioè una "scatola nera" con la quale l'interazione avviene solo e solamente tramite i metodi definiti dall'interfaccia. Il punto è dare delle funzionalità agli utenti nascondendo i dettagli legati alla loro implementazione.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Ereditarietà

Il meccanismo dell'ereditarietà permette di derivare nuove classi a partire da quelle già definite realizzando una gerarchia di classi. Una classe derivata attraverso l'ereditarietà (sottoclasse o classe figlia) mantiene i metodi e gli attributi delle classi da cui deriva (classi base, superclassi o classi madre); inoltre, può definire i propri metodi o attributi, e ridefinire il codice di alcuni dei metodi ereditati tramite un meccanismo chiamato overriding.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Polimorfismo

Nella programmazione ad oggetti, con il nome di polimorfismo si indica il fatto che lo stesso codice eseguibile può essere utilizzato con istanze di classi diverse, aventi una *superclasse* comune.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

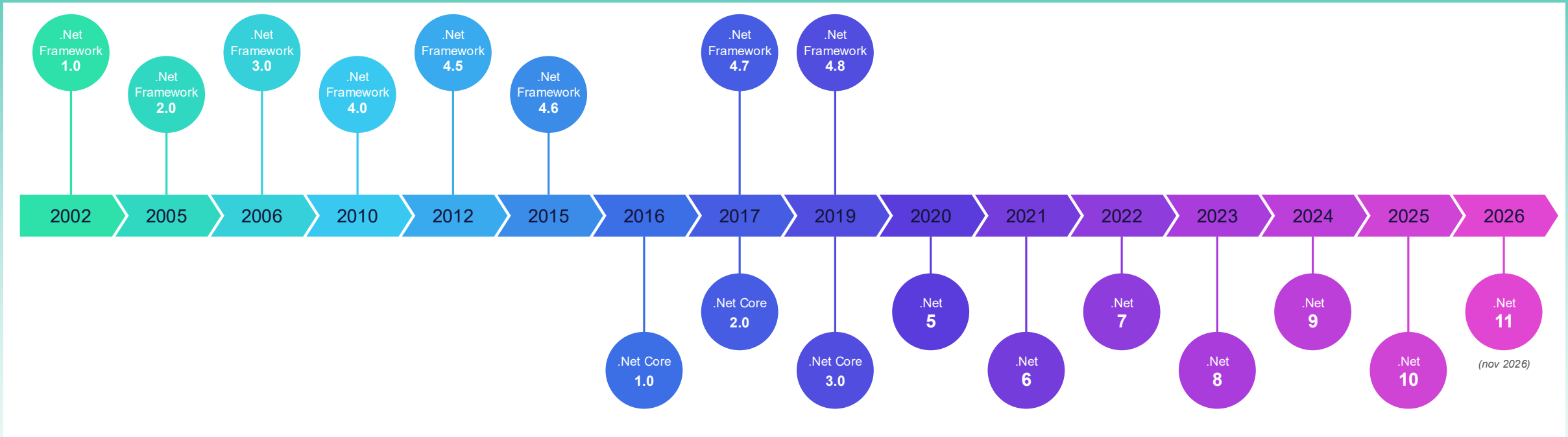
Microsoft .NET

Microsoft .NET è una piattaforma di sviluppo ideata e sviluppata da **Microsoft** che mette a disposizione varie funzionalità come **il supporto per più linguaggi di programmazione** (coprendo tutti i paradigmi di programmazione: object oriented, procedurali, funzionali, imperativi, user oriented, ecc.), modelli di programmazione asincroni e simultanei, interoperabilità, consentendo l'esecuzione su più piattaforme ed in presenza di scenari applicativi variegati.

Wikipedia

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

.NET history



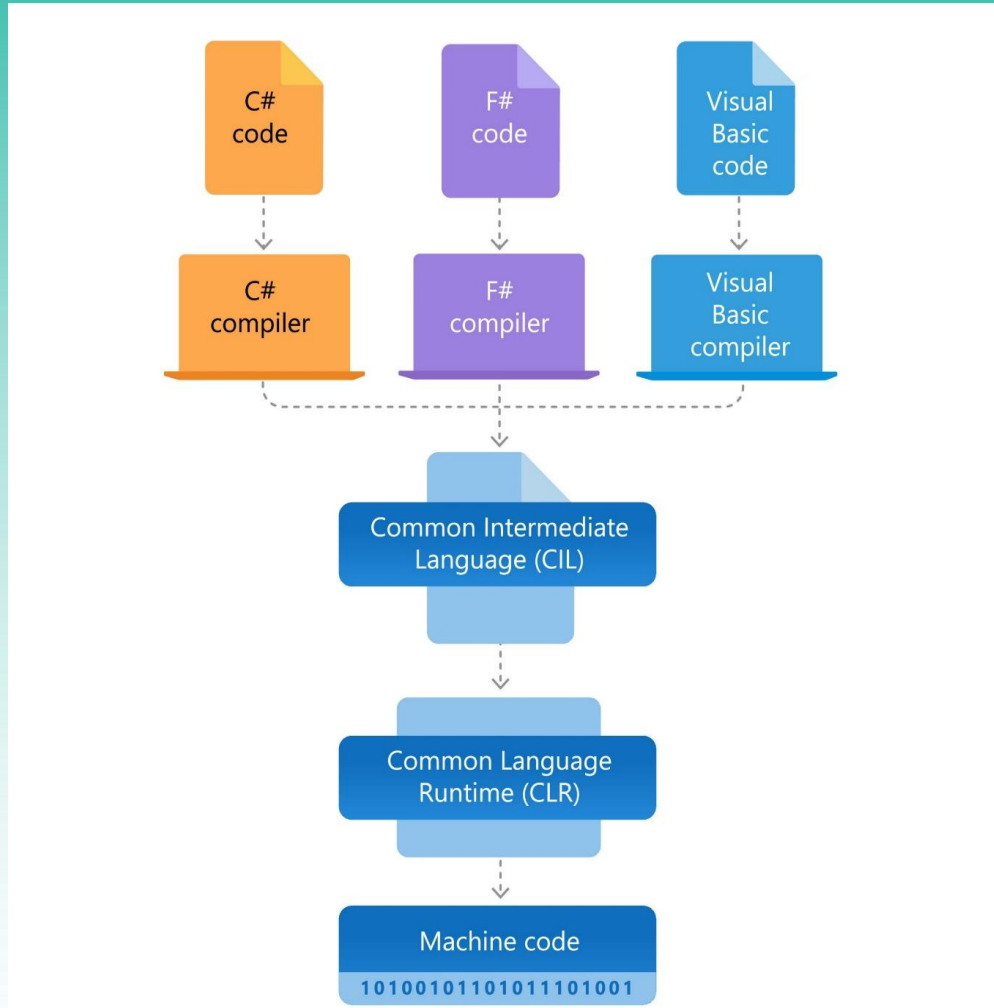
COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Linguaggio C#

- Prima versione commerciale rilasciata da Microsoft nel 2002 assieme alla prima versione di .NET
- Linguaggio derivato dal linguaggio C (linguaggio compilato)
- Linguaggio fortemente tipizzato
- Ultima versione: 14 (Giugno 2025)
- Sviluppo desktop, mobile e web
- Sviluppo di applicazioni aziendali, web, giochi, app mobile, ecc.
- Linguaggio prima compilato e poi interpretato
- IDE
  - Visual Studio (community o enterprise)
  - Visual Studio Code
  - Altri (Rider, ecc.)

Dal codice all'esecuzione

1. Scrittura del codice sorgente in linguaggio C# (o altri linguaggi .NET);
2. Compilazione tramite compilatore C#, che produce un codice intermedio (CIL - Common Intermediate Language), non ancora eseguibile, ma indipendente dal linguaggio, ovvero ottenibile anche da sorgenti scritti in altri linguaggi di programmazione;
3. Tramite il CLR – Common Language Runtime, il compilatore di .NET traduce il linguaggio intermedio in linguaggio nativo eseguibile (linguaggio macchina).



Assembly

Un programma C# che è stato compilato nel formato intermedio del CIL viene chiamato Assembly.

Un Assembly è una raccolta di tipi creati per essere usati insieme e per formare un'unità logica di funzionalità.

Gli Assembly hanno la forma di file eseguibili (*.exe*) o libreria a collegamento dinamico (*.dll*) e sono i blocchi predefiniti delle applicazioni .NET.

Gli Assembly forniscono al Common Language Runtime le informazioni necessarie per riconoscere le implementazioni dei tipi.

Tipologie:

- Process Assembly: programma completo ed autonomo;
- Library Assembly: contiene codice che può essere utlizzato da altri programmi.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE TURISMO E NUOVE TECNOLOGIE

Struttura di un programma C#

```
using System;

//NAMESPACE
namespace MyNamespace {

    //CLASS
    class MyClass {

        //METHOD
        static void MyMethod(string variable) {

            //STATEMENT
            //keyword   identifier   operator   literal   separator
            //int       i            =          0         ;
            int i = 0;

            while(i < 15)
            //CODE BLOCK
            {
                Console.WriteLine(variable);
                i++;
            }
        }
    }
}
```

- Keyword: parola chiave predefinita nel linguaggio, che ha un particolare significato per il compilatore;
- Identifier: nome che attribuiamo a variabili, metodi, classi, namespace per identificarli;
- Operator: simbolo che indica al compilatore di svolgere una certa operazione;
- Literal: valori dei tipi conosciuti dal linguaggio, come numeri o stringhe;
- Separator: caratteri che segnalano l'inizio o la fine di parti di codice;
- Statement: istruzione, secondo le regole di sintassi del linguaggio, che indica al compilatore di eseguire un compito preciso;

Struttura di un programma C#

```
using System;

//NAMESPACE
namespace MyNamespace {

    //CLASS
    class MyClass {

        //METHOD
        static void MyMethod(string variable) {

            //STATEMENT
            //keyword   identifier   operator   literal   separator
            //int       i            =          0         ;
            int i = 0;

            while(i < 15)
            //CODE BLOCK
            {
                Console.WriteLine(variable);
                i++;
            }
        }
    }
}
```

- Code Block: blocco di codice, cioè collezione di statement, tipicamente definito all'interno di parentesi graffe;
- Method: contenitore di blocchi di codice, ai quali viene assegnato un identificatore, in modo da poter essere richiamato in altre parti del programma. Può prendere in input dei parametri e può restituire un valore di ritorno.
- Class: è un tipo in C# che può contenere variabili e metodi al suo interno;
- Namespace: contenitori di classi e altri tipi (come strutture, enumerazioni, interfacce);
- Program: file sorgenti che contengono namespace. I programmi compilati vengono chiamati Assembly.

Prerequisiti

- Installazione .NET SDK (versione .NET 10)
  https://dotnet.microsoft.com/it-it/download
- Visual Studio Code
  https://code.visualstudio.com/download
- Estensione C#
  https://marketplace.visualstudio.com/items?itemName=ms-dotnettools.csharp

Dotnet CLI

.NET SDK include un’interfaccia a riga di comando molto utile in Visual Studio Code.

La Dotnet CLI permette di:

- Creare un nuovo progetto
- Creare una nuova soluzione
- Gestire le dipendenze
- Avviare un’applicazione
- Pubblicare un’applicazione

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Dotnet CLI – Comandi principali

- Creare un nuovo progetto .NET 10
  - `dotnet new`
- Compilare un progetto (viene creato l'assembly: nomeProgetto.dll)
  - `dotnet build`
- Avviare un progetto (viene prima eseguita la compilazione)
  - `dotnet run`
- Avviare un progetto con hot reload
  - `dotnet watch`
- Aggiungere una dipendenza
  - `dotnet add package nomeDipendenza`
- Ripristinare le dipendenze
  - `dotnet restore`

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione1 - Hello World

Console Application

Un'applicazione **console** o **programma a riga di comando** è un programma progettato per essere utilizzato tramite un'interfaccia utente solo testo.

- Su Visual Studio Code aprire un nuovo terminale dal menu Terminal - New Terminal (CTRL+ò)
- Spostarsi nella directory in cui si vuole creare il nuovo progetto, ad esempio per spostarsi nella cartella CorsoITS:
  - `cd <cartella_dei_progetti_its> + Invio`
  - `cd CorsoITS + Invio`
- Creare un nuovo progetto .NET 10 di tipo console denominato Esercitazione1
  - `dotnet new console -n Esercitazione1 -f net10.0`

Esercitazione1 - Hello World

Il terminale avvisa della creazione del progetto.

Aprire il progetto dal menù File/Open Folder.

All’interno della cartella del progetto sono presenti:

- bin: directory che conterrà i file binari, che costituiscono il codice eseguibile effettivo dell'applicazione, quindi gli assembly (alla creazione del progetto non contiene files);
- obj: directory che contiene gli output di compilazione intermedi creati durante il processo di compilazione;
- Esercitazione1.csproj: è il file di progetto C#;
- Esercitazione1.sln: è il file della soluzione. Una soluzione è una struttura per organizzare i progetti in Visual Studio;
- Program.cs: è il file che contiene il codice di avvio del progetto.

Esercitazione1 - Hello World

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework
>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

Esercitazione1.csproj
Il file csproj è il file di progetto, è un file XML contenente tutte le informazioni e le istruzioni richieste da MSBuild (Microsoft Build Engine) per compilare il progetto, ad esempio il contenuto da includere, i requisiti della piattaforma, le informazioni sul controllo delle versioni, le impostazioni del server Web o del server di database e le attività da eseguire.

Esercitazione1 - Hello World

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

Program.cs
Il file Program.cs è il file che contiene il codice di avvio dell'applicazione.

Alla creazione del progetto contiene un'istruzione che, se eseguita, stampa a console la scritta 'Hello, World!'.

In questo esempio, viene invocato il metodo WriteLine della classe Console.
La classe Console è contenuta in una libreria predefinita di .NET.

Fino a C# 8 il metodo Main rappresentava il punto d'ingresso .

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione1 - Hello World

Nel terminale, posizionandosi nella cartella del progetto Esercitazione1, digitare:

- `dotnet run`

Il comando esegue il codice sorgente, dopo aver eseguito la compilazione.

Nel terminale viene visualizzata la stringa ‘Hello, World!’

Nella cartella bin\Debug\net8.0 sono ora presenti dei files di output, tra cui Esercitazione1.dll, cioè l’assembly dell’applicazione.

I files temporanei vengono inseriti nella directory obj.

Nel terminale spostarsi dentro la cartella Esercitazione1\bin\Debug\net8.0 e digitare:

- `dotnet Esercitazione1.dll`

Nel terminale viene visualizzata la stringa ‘Hello, World!’

Esercitazione2 - Hello World, metodo Main

- Su Visual Studio Code aprire un nuovo terminale dal menu Terminal - New Terminal
- Nel terminale, spostarsi nella directory in cui si vuole creare il nuovo progetto, ad esempio, dalla radice, per spostarsi nella cartella CorsoITS del Desktop;
- Creare un nuovo progetto .NET 10 di tipo console denominato Esercitazione2
  - `dotnet new console -n Esercitazione2 **--use-program-main** -f net10.0`

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione2 - Hello World, metodo Main

```
namespace Esercitazione2;

class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```

Nel namespace Esercitazione2, all'interno della classe Program, è presente il metodo statico Main, che invoca il metodo WriteLine della classe Console.
La classe Console è contenuta in una libreria predefinita di .NET, del namespace System.

A partire da C# 9 il codice è stato semplificato (Esercitazione1).

Esercitazione2Plus - Hello World, Main Args

```
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
        if(args.Length > 0)
        /* Controllo se ci sono argomenti da linea di comando */
        {
            Console.WriteLine("Arguments provided:");
            foreach (var arg in args)
            {
                Console.WriteLine(arg);
            }
            // TODO Restituire il 2' argomento
        }
        else
        {
            Console.WriteLine("No arguments provided.");
        }
    }
}
```

Il parametro string[] args del metodo Main permette di ricevere argomenti da riga di comando:

tutto ciò che viene scritto dopo -- nel comando dotnet run viene passato al programma come input, utile per personalizzare l'esecuzione.

Esercitazione3 - Dependency

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo console denominato Esercitazione3
  - `dotnet new console -n Esercitazione3`
- Aggiungere al progetto Esercitazione3 una dipendenza (es. Humanizer)
  - `dotnet add package Humanizer`
- Che cos'è Humanizer?

Esercitazione3 - Dependency

```xml
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Humanizer" Version="2.14.1" />
  </ItemGroup>

</Project>
```

Esercitazione3.csproj
Nel file csproj è presente un nuovo tag xml (<ItemGroup>), che contiene il riferimento al package 'Humanizer' e la sua versione.
I riferimenti ai pacchetti, tramite <PackageReference>, specificano le dipendenze dei pacchetti direttamente all'interno dei file di progetto.
All'interno della cartella bin sono presenti le dll relative a Humanizer.

Esercitazione3 - Dependency

Humanizer è un package per la manipolazione e la visualizzazione di stringhe, enum, date, orari, intervalli di tempo, numeri e quantità.

Firma del metodo Pluralize:

string string.Pluralize([bool inputIsKnownToBeSingular = true])

E’ un metodo che restituisce il plurale della stringa su cui è applicato.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione3 - Dependency

```
using Humanizer;

Console.WriteLine("Woman".Pluralize());
```

Program.cs

- Modificare il Program.cs e inserire il codice come in figura sopra che richiama il metodo Pluralize() e stampa a console il risultato restituito dal metodo;
- Eseguire l'applicazione.

A console viene visualizzata la scritta: *Women*

Esercitazione4 – Tipi di dato e classi

Tipi di dato

I tipi valore e i tipi riferimento sono le due categorie principali di tipi C#.

- Una variabile di un tipo valore contiene un'istanza del tipo. Per impostazione predefinita, per l'assegnazione, il passaggio di un argomento a un metodo e la restituzione di un risultato del metodo vengono copiati i valori delle variabili.
- Le variabili dei tipi di riferimento archiviano i riferimenti ai relativi dati. Con i tipi di riferimento, due variabili possono fare riferimento allo stesso oggetto. Di conseguenza le operazioni su una variabile possono influire sull'oggetto a cui fa riferimento l'altra variabile.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione4 – Tipi di dato e classi

Ogni parola chiave di tipo C# è un alias per il tipo .NET corrispondente.

Alcuni Tipi valore predefiniti in C#

Parola chiave del tipo C#	Tipo .NET
bool	System.Boolean
byte	System.Byte
sbyte	System.SByte
char	System.Char
decimal	System.Decimal
double	System.Double
float	System.Single
int	System.Int32
uint	System.UInt32
long	System.Int64
ulong	System.UInt64
short	System.Int16
ushort	System.UInt16

Alcuni Tipi di riferimento predefiniti in C#

Parola chiave del tipo C#	Tipo .NET
object	System.Object
string	System.String

Alcuni Valori predefiniti dei tipi in C#

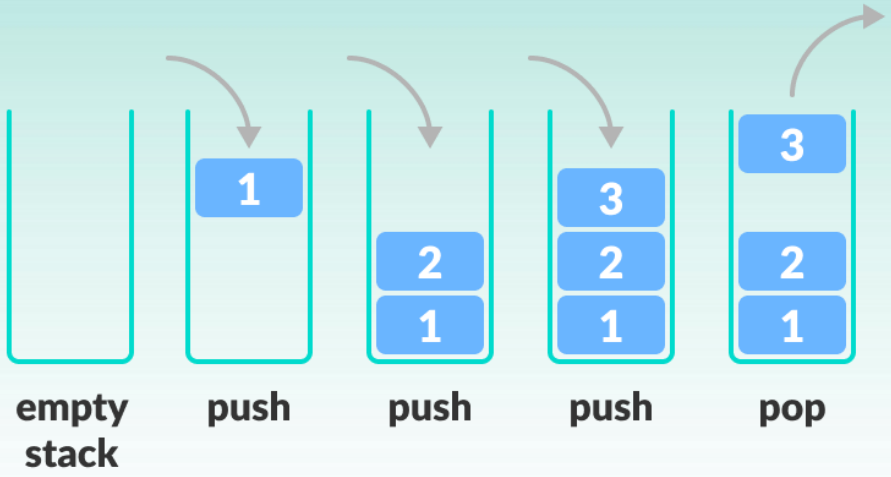
Type	Default value
Qualsiasi tipo riferimento	null
Qualsiasi tipo numerico	0 (zero)
bool	false

Esercitazione4 – Tipi di dato e classi

Stack e Heap

Nei value e reference types in C Sharp quando un programma C# va in esecuzione l'ambiente di runtime crea due aree di memoria ben distinte, lo stack e lo heap. Iniziamo a parlare dello stack.

Lo stack (pila in italiano) è un'area di memoria molto veloce, una struttura di tipo LIFO (Last In First Out) cioè l'ultimo valore immagazzinato è il primo ad uscire.



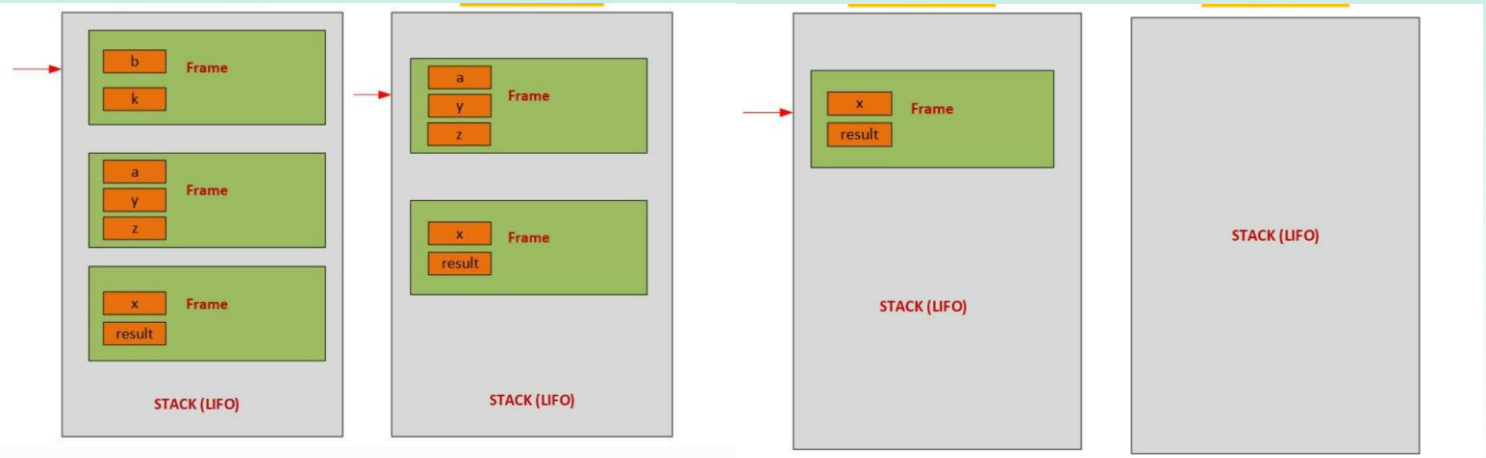
COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione4 – Tipi di dato e classi

Stack e Heap

Lo stack tiene traccia del flusso di esecuzione del programma. Abbiamo visto che in una applicazione console il metodo Main è il primo ad essere eseguito. Quando il programma entra in questo metodo nello stack viene creato un cosiddetto frame che memorizza tutti i parametri del metodo.

Quando arriva la call al FirstMethod l'esecuzione del programma salta a questo metodo. Sullo stack viene creato un nuovo frame in cui vengono immagazzinati il parametro di input a e le due variabili locali y e z. Il frame appena creato occupa la cima dello stack Figura 2. Quando viene chiamato SecondMethod viene creato nuovamente un frame che memorizza tutti parametri e variabili di SecondMethod Figura 3 e l'esecuzione prosegue con l'esecuzione di quest'ultimo. Al termine di SecondMethod si risale la cima dello stack in particolare tutte i parametri e le variabili di questo metodo vengono cancellati dalla memoria e si passa di nuovo ad eseguire FirstMethod e così via fino all'ultima istruzione del metodo Main che termina il programma (Figura 4).



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

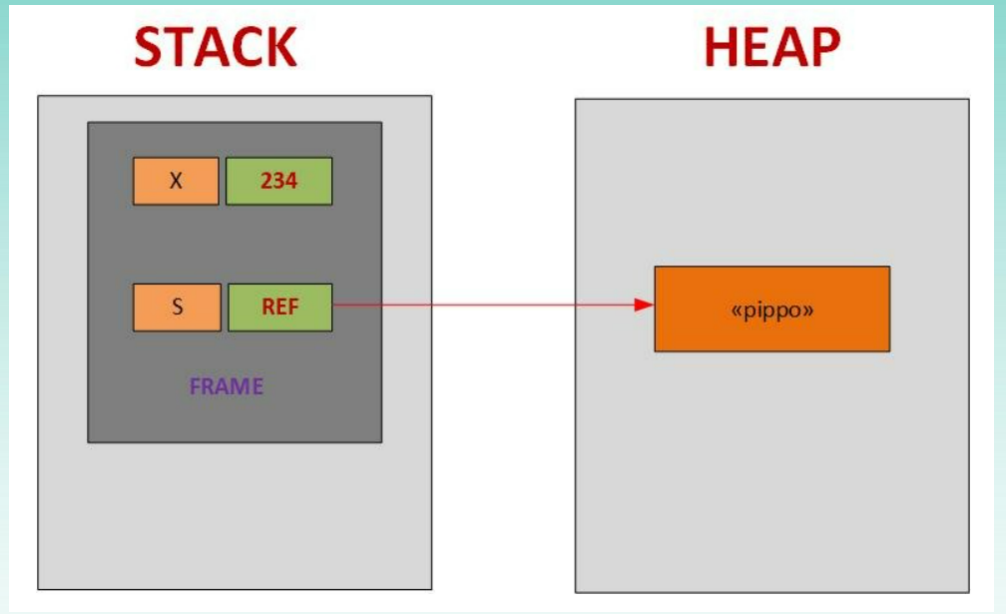
Esercitazione4 – Tipi di dato e classi

Stack e Heap

L'Heap (catasta mucchio in italiano) a differenza dello stack è un'area di memoria in cui non viene tracciata l'esecuzione del programma, questo è compito dello stack ma è un'area di memoria disordinata in cui vengono memorizzati gli oggetti per un certo tempo. È compito del Garbage Collector quando non ci sono più riferimenti ad un oggetto liberare la memoria, ma questo lo vedremo meglio parlando dei Value Types e Reference Types.

Vediamo con una figura come l'ambiente di runtime gestisce il tipo int (Value Types) e il tipo string (Reference Types).

Per quanto riguarda il comportamento dei value types come ad esempio un int vediamo che nello stack oltre alla variabile x viene memorizzato direttamente il valore 234 ossia la variabile x contiene o per meglio dire punta direttamente il valore 234. Per quanto riguarda invece le stringhe che abbiamo già incontrato, nello stack viene memorizzata la variabile e il riferimento alla stringa ossia nel reference è contenuta la locazione di memoria che si trova nello heap e che contiene il valore "pippo". Questa è la grossa differenza tra un value types che memorizzano direttamente il valore insieme alla variabile nello stack e i reference types che nello stack o meglio nel frame creato dal runtime memorizzano una variabile che punta il riferimento all'oggetto memorizzato sullo Heap.



Esercitazione4 – Tipi di dato e classi

Supponiamo che il nostro programma esegua il metodo Main nel quale abbiamo dichiarato due variabili x e y di tipo intero e assegniamo il valore di x ad y. Il runtime come sai crea un frame nello stack e all'interno del frame memorizza le variabili x e y insieme ai loro valori.

int x = 120;

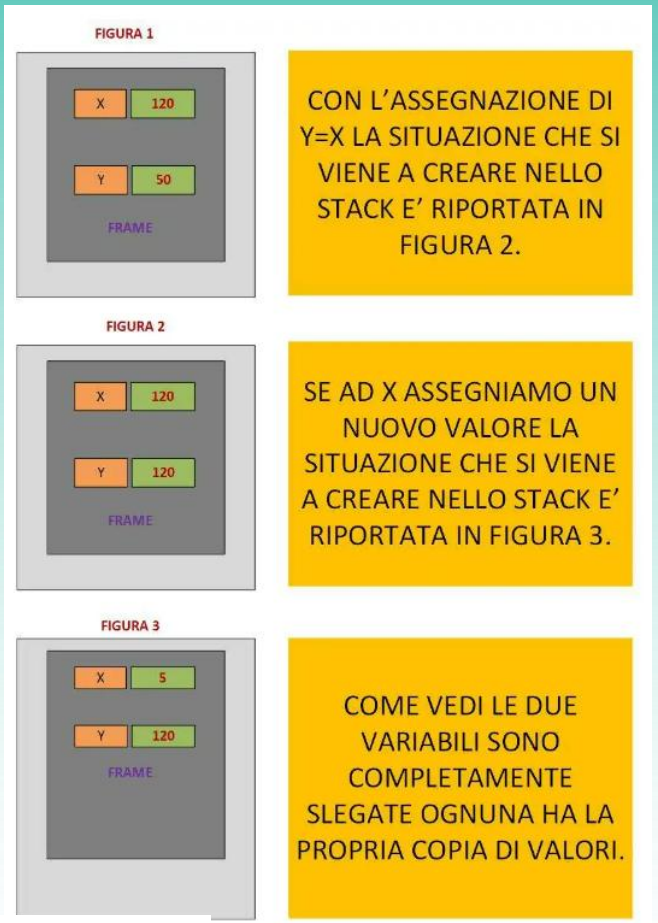
int y = 50;

y=x;

x=5;

vediamo con una figura la situazione che si crea nello stack.

Quando il flusso del programma nel Main assegna il valore di x ad y vengono create due copie separate per il valore literal 120 una per x e l'altra per y. Quando riassegniamo il valore ad x (x=5) si vede che y mantiene il precedente valore e x assume 5. Ciò sta a dimostrare che per i value types le due variabili sono completamente slegate.



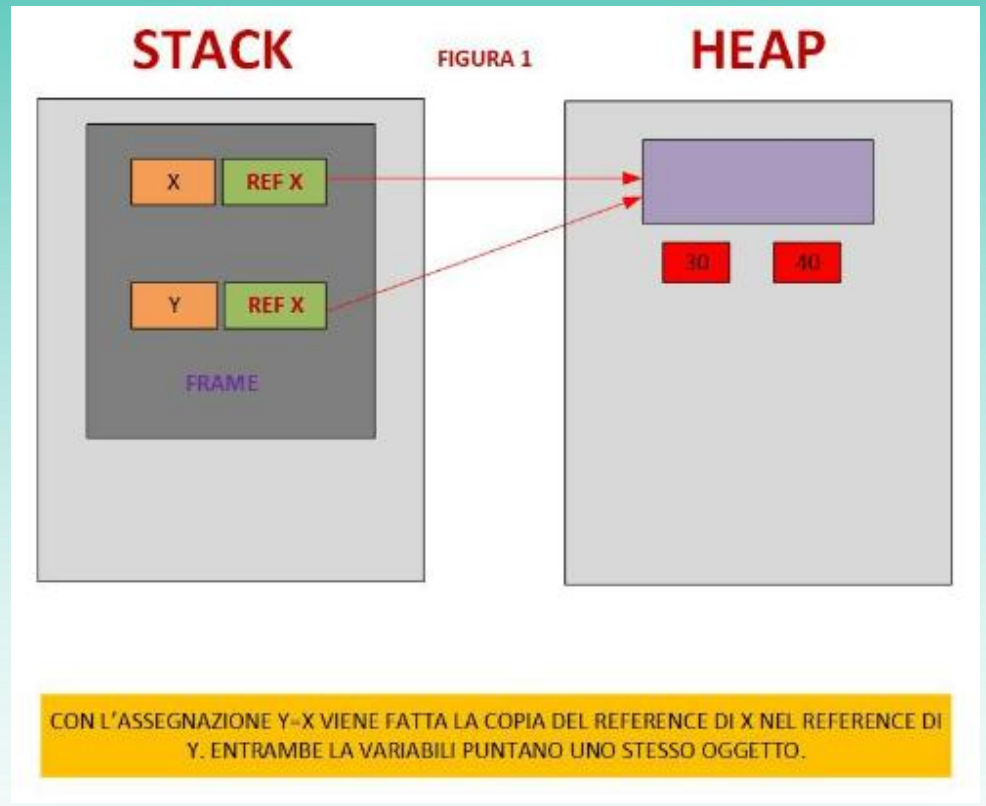
Esercitazione4 – Tipi di dato e classi

Supponiamo che nel nostro metodo Main dichiariamo un array di int, gli array li vedremo successivamente. Un array è una collezione di oggetti di un certo tipo la sintassi per dichiarare un array è riportata più in basso. Poi supponiamo di creare un secondo array y ed assegnarlo a x; Infine modifichiamo il primo elemento di x. Vediamo con una figura come si presenta la situazione in memoria.

int [] x = **new** [] **{30,40};**

int [] **y = x;**

x[0] = 10;

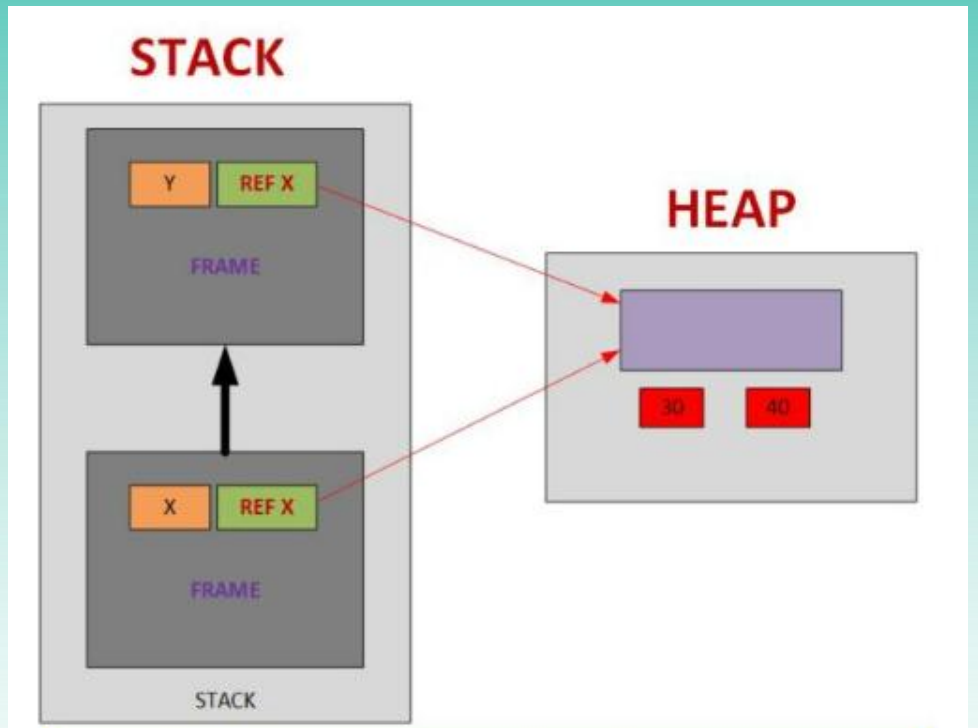


COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione4 – Tipi di dato e classi



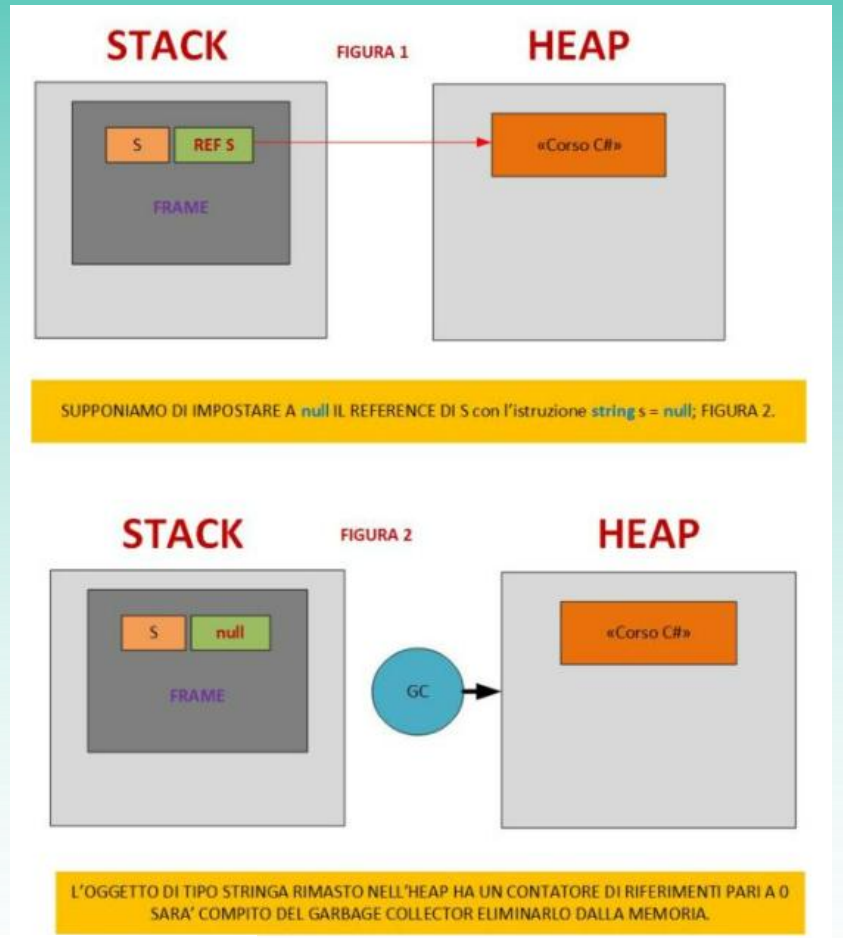
NEL PASSAGGIO DI PARAMETRI PER VALORE QUANDO FIRST METHOD CHIAMA SECOND METHOD PASSANDO L'INTERO 120, ANCHE SE Y MODIFICA QUESTO VALORE IL VALORE DI X AL RITORNO RIMANE INTATTO.



CON IL PASSAGGIO DI PARAMETRI PER RIFERIMENTO QUELLO CHE VIENE PASSATO DAL CHIAMANTE E' UN REFERENCE QUINDI SE NEL METODO CHIAMATO VIENE MODIFICATA LA VARIABILE Y QUESTO SI RIPERCUOTE ANCHE SU X CHE AL RITORNO DAL METODO VEDE MODIFICATO IL SUO VALORE.

Esercitazione4 – Tipi di dato e classi

La keyword null rimuove il riferimento dallo stack che conteneva la locazione di memoria dell'oggetto puntato sull'heap. Se l'oggetto rimane orfano sull'heap, cioè non possiede più riferimenti attivi esiste un meccanismo in C# chiamato garbage collector che esegue una scansione e una pulizia di tutti gli oggetti rimasti orfani nello heap. Se non ci fosse questo meccanismo che è automatico ben presto si arriverebbe ad un memory leak. Vediamo di chiarire il concetto con una immagine.



Esercitazione4 – Tipi di dato e classi

Classe

Una classe è un tipo di riferimento che è possibile derivare direttamente da un'altra classe e che viene derivato in modo implicito da System.Object.

La classe definisce le operazioni che un oggetto (un'istanza della classe) può eseguire (metodi) e i dati che l'oggetto contiene (campi).

Una classe può implementare un numero qualsiasi di interfacce, ma può ereditare solo da una classe di base, oltre che da System.Object, da cui tutte le classi ereditano in modo implicito.

Tutte le classi devono avere almeno un costruttore, per l'inizializzazione di nuove istanze della classe. Se non si definisce in modo esplicito un costruttore, la maggior parte dei compilatori specifica automaticamente un costruttore senza parametri.

Esercitazione4 – Cicli

In alcune situazioni c'è la necessità di ricorrere a dei cicli o istruzioni di iterazione. In C#, questi possono essere costruiti grazie ai cicli: while, for e foreach. Le istruzioni di iterazione possono essere utilizzate per creare cicli per fare in modo che le istruzioni incorporate siano eseguite un determinato numero di volte, in base ai criteri di terminazione del ciclo.

La sintassi di un ciclo while è la seguente:

```
while (condizione booleana)
{
    //ISTRUZIONI
}
```

La condizione booleana è una normale espressione che restituisce true o false. È importante notare che se tale condizione è false prima di iniziare il ciclo (la condizione viene verificata prima) allora il ciclo while non verrà mai eseguito. While itera il blocco di codice racchiuso tra le parentesi graffe fintantoché la condizione è true.

Esercitazione4 – Rami e Cicli

La sintassi del ciclo do-while è la seguente:

```
do

{

    //ISTRUZIONI

} while (condizione booleana)
```

La variante rispetto al ciclo while è che il loop almeno una volta viene eseguito in quanto la condizione booleana è verificata alla fine del blocco di codice.

Esercitazione4 – Rami e Cicli

La sintassi del ciclo for è la seguente:

```
for (condizione iniziale; condizione di test; istruzione di fine loop)

{

        //ISTRUZIONI

}
```

Come vedi il ciclo for è formato nell'intestazione da tre parti, separate da un punto e virgola. La condizione iniziale in cui generalmente viene dichiarata e inizializzata una variabile di controllo, la condizione di test rappresentata da una condizione booleana e l'istruzione di fine loop solitamente rappresentata dall'incremento o decremento della variabile di controllo.

Esercitazione4 – Rami e Cicli

L'ultimo ciclo da esaminare è il foreach. Questo tipo di ciclo serve ad iterare oggetti. Esegue un'istruzione o un blocco di istruzioni per ogni elemento di una istanza di un tipo. Ad esempio:

```
string myVar = "pippo";

foreach(char c in myVar)

{

    Console.WriteLine(c);

}
```

Stamperà tutti i singoli caratteri che si trovano nella stringa pippo.

Mettiamo in pratica Array e Liste E Cicli

Creare una nuova applicazione a console che:

- definisca un array di stringhe, lo valorizzi, e conti quante volte un valore specifico compare nell'array (inserire una stampa a console di un esempio);
- definisca una matrice quadrata di numeri interi, lo valorizzi, e calcoli la somma degli elementi presenti sulla diagonale principale (inserire una stampa a console di un esempio);
- definisca una lista di stringhe, lo valorizzi, e utilizzi un ciclo 'foreach' per cercare una parola specifica nella lista e stampare se la parola della ricerca è presente (inserire una stampa a console di un esempio);
- definisca una lista di numeri interi, lo valorizzi, e utilizzi un ciclo 'foreach' per calcolare la media di tutti gli elementi della lista (inserire una stampa a console di un esempio).

Esercitazione5 – Istruzioni di primo livello

Programmi senza metodo Main

Non è necessario includere in modo esplicito un metodo Main in un progetto di applicazione console.

È invece possibile usare la funzionalità *istruzioni di primo livello* per ridurre al minimo il codice da scrivere.

In questo caso, il compilatore genera una classe Program.

- Un progetto può avere un solo file con istruzioni di primo livello;
- Se si includono direttive using devono venire prima nel file;
- Le istruzioni di primo livello sono in modo implicito nello spazio dei nomi globale;
- Un file con istruzioni di primo livello può contenere anche spazi dei nomi e definizioni dei tipi, ma devono venire dopo le istruzioni di primo livello;
- Per restituire un valore al termine dell'applicazione, usare l'istruzione return come in un metodo Main che restituisce un valore.

Esercitazione6 – Object Oriented

Un linguaggio di programmazione è definito *ad oggetti* quando, usando la sintassi nativa del linguaggio, permette di implementare:

- incapsulamento;
- ereditarietà;
- polimorfismo.

Esercitazione6 – Incapsulamento

Per alterare lo stato interno dell'oggetto, è necessario invocarne i metodi pubblici, ed è questo lo scopo principale dell'incapsulamento.

Infatti, se gestito opportunamente, esso permette di vedere l'oggetto come una *black-box*, cioè una "scatola nera".

Il punto è dare delle funzionalità agli utenti nascondendo i dettagli legati alla loro implementazione.

Esercitazione6 – Ereditarietà

L'ereditarietà consente di creare nuove classi che riutilizzano, estendono e modificano il comportamento definito in altre classi.

La classe i cui membri vengono ereditati è denominata *classe base*, mentre la classe che eredita i membri è denominata *classe derivata*.

Una classe derivata può avere una sola classe di base diretta.

L'ereditarietà, tuttavia, è transitiva. Se ClassC eredita da ClassB e ClassB eredita da ClassA allora ClassC eredita da ClassA.

Esercitazione6 – Polimorfismo

Polimorfismo è una parola che deriva dal greco e significa "multiforme".

Il polimorfismo presenta due aspetti distinti:

- In fase di esecuzione, oggetti di una classe derivata possono essere trattati come oggetti di una classe base.
- Le classi base possono definire e implementare metodi *virtuali* e le classi derivate possono eseguirne l'override, ovvero ne forniscono una propria definizione e implementazione. Durante la fase di esecuzione, quando il codice client chiama il metodo, CLR cerca il tipo in fase di esecuzione dell'oggetto e richiama quell'override del metodo virtuale.

Mettiamo in pratica

Creare una nuova applicazione a console che contenga:

- una classe Persona dotata dei campi:
  1. AnnoDiNascita,
  2. Nome,
  3. Cognome
- una classe Studente come estensione di Persona; i campi aggiuntivi di Studente rispetto a Persona sono:
  1. Matricola,
  2. Università
- la classe Docente che estende Persona; i campi aggiuntivi di Docente rispetto a Persona sono:
  1. Materia
  2. Salario

Ogni classe dovrà avere i metodi per leggere e valorizzare i campi e dovranno essere dotate del metodo toString().

Nel Program.cs creare un oggetto per ogni tipo di classe in cui vengano valorizzati tutti i dati e stampare i dati delle persone inserite.

Infine modificare il codice fiscale dello Studente e stampare a video solo il codice fiscale modificato.

Web API

API

Il termine API, acronimo di Application Programming Interface (interfaccia di programmazione delle applicazioni), indica un insieme di definizioni e protocolli per la creazione e l'integrazione di applicazioni software.

Talvolta vengono concepite come una forma di contratto, con una documentazione che rappresenta un accordo tra le parti: se la parte A invia una richiesta remota strutturata in un determinato modo, il software della parte B risponderà in un altro modo determinato.

Le API sono state introdotte agli albori dell'era informatica, molto prima della comparsa dei personal computer. A quell'epoca, un'API era in genere utilizzata come libreria per un sistema operativo. Era a livello locale rispetto al sistema in cui operava, sebbene a volte passasse messaggi tra mainframe. Dopo circa 30 anni, le API sono emerse dai loro ambienti locali. Nei primi anni 2000 si trasformarono in un'importante tecnologia per l'integrazione remota dei dati.

Le API remote sono concepite per interagire attraverso una rete di comunicazione. Si chiamano API remote perché le risorse gestite da tali API si trovano all'esterno del computer che invia la richiesta. Poiché il canale di comunicazione maggiormente utilizzato è Internet, quasi tutte le API vengono progettate in base agli standard web.

Queste ultime utilizzano in genere l'HTTP per richiedere messaggi e fornire una definizione della struttura dei messaggi di risposta. I messaggi di risposta assumono in genere la forma di file XML o JSON, due formati diffusi perché presentano i dati in modo da consentire alle altre app di gestirli facilmente.

Web API (Web service vs Web API)

L'utilizzo del Web come piattaforma di programmazione è un'idea che si è diffusa intorno al 2000 con l'introduzione dei Web service, cioè di un meccanismo per l'interoperabilità tra applicazioni basato sulle tecnologie e gli standard definiti per il Web.

I Web service consentono di far interagire due applicazioni indipendentemente dal sistema operativo su cui girano e dal linguaggio di programmazione utilizzato. Tecnicamente il meccanismo di interazione è basato sul protocollo SOAP, su un insieme di regole che definiscono la struttura dei messaggi scambiati dalle applicazioni e tutta una serie di accorgimenti per garantire sicurezza, affidabilità, corretta gestione degli eventi, ecc. L'approccio è sostanzialmente basato su una definizione di funzione richiamabile via Web.

Un approccio alternativo ai Web service basati su SOAP, che è stato proposto negli stessi anni ma è stato riscoperto soltanto negli ultimi tempi, è l'approccio noto come REST (Representational State Transfer) e basato su una serie di principi che un'applicazione deve rispettare. In sintesi, nella visione RESTful un Web service non definisce una funzione richiamabile da remoto ma mette a disposizone delle risorse su cui è possibile effettuare le classiche operazioni CRUD sfruttando i metodi del protocollo HTTP.

Due visioni diverse del Web come piattaforma di elaborazione che sempre più spesso vengono distinte parlando semplicemente di Web service quando si fa riferimento a quelli basati si SOAP e di Web API quando ci ispira al modello REST.

Web API (SOAP vs REST)

Vista la continua diffusione delle API web, è stato sviluppato un protocollo specifico con lo scopo di standardizzare lo scambio delle informazioni, denominato Simple Object Access Protocol, o protocollo SOAP. Le API progettate con il protocollo SOAP usano il linguaggio XML come formato del messaggio e ricevono le richieste tramite HTTP o SMTP. Il protocollo SOAP facilita la condivisione delle informazioni per le app eseguite su ambienti diversi o scritte in linguaggi differenti.

È stata introdotta anche la specifica Representational State Transfer (REST). Le API web che rispettano i vincoli architetturali REST vengono definite API RESTful.

La differenza tra REST e SOAP è sostanziale: il SOAP è un protocollo, mentre il REST è un tipo di architettura e ciò implica l'assenza di uno standard ufficiale per le API web RESTful.

Web API (API REST)

Le API sono definibili RESTful se rispettano i sei vincoli di un sistema RESTful:

- Architettura client-server: l'architettura REST è costituita da client, server e risorse e gestisce le richieste tramite il protocollo HTTP.
- Stateless: nessun contenuto client è archiviato nel server tra le richieste. Le informazioni relative allo stato della sessione sono invece contenute nel client.
- Supporto cache: il caching può eliminare la necessità di alcune interazioni client-server.
- Sistema a livelli: le interazioni client-server possono essere mediate da livelli aggiuntivi, i quali possono offrire altre funzionalità, come bilanciamento del carico, condivisione della cache o sicurezza.
- Codice on demand (opzionale): i server possono ampliare la funzionalità di un client trasferendo del codice eseguibile.
- Interfaccia uniforme: è il vincolo principale per la progettazione di API RESTful e prevede 4 aspetti:
    1. Identificazione delle risorse nelle richieste: le risorse vengono identificate nelle richieste e vengono distinte dalle rappresentazioni restituite al client.
    2. Manipolazione delle risorse tramite le rappresentazioni: i client ricevono file che rappresentano le risorse e che devono contenere le informazioni necessarie per consentirne la modifica o l'eliminazione.
    3. Messaggi autodescrittivi: ogni messaggio restituito a un client contiene le informazioni necessarie per descrivere come il client deve elaborare l'informazione.
    4. Ipermedia come motore dello stato dell'applicazione: accedendo alla risorsa, il client REST deve poter individuare, attraverso hyperlink, tutte le altre azioni disponibili al momento.

Benché sembrino numerosi, questi vincoli sono molto più semplici rispetto a un protocollo prescritto e, proprio per questo, le API RESTful sono molto più diffuse dei metodi SOAP.

Negli ultimi anni, la specifica OpenAPI si è imposta come standard comune per la definizione delle API REST, poiché consente agli sviluppatori di realizzare interfacce API REST indipendenti dal linguaggio e facilmente comprensibili agli utenti.

Web API

API Web di base ASP.NET

ASP.NET Core è un framework open source, multipiattaforma e ad alte prestazioni per la creazione di app moderne, abilitate al cloud e connesse a Internet.

Con ASP.NET Core si può:

- Creare app e servizi web, app per backend per dispositivi mobili
- Utilizzare strumenti di sviluppo su Windows, macOS e Linux
- Distribuire sul cloud o in locale
- Eseguire l'app su .NET

Web API (openAPI)

Supporto openAPI nelle app per le API principali di ASP.NET

La specifica OpenAPI è uno standard indipendente dal linguaggio di programmazione per la documentazione delle API HTTP. Questo standard è supportato in API minime tramite una combinazione di API predefinite e librerie open source. L'integrazione OpenAPI in un'applicazione presenta tre aspetti chiave:

- Generazione di informazioni sugli endpoint nell'app.
- Raccolta delle informazioni in un formato corrispondente allo schema OpenAPI.
- Esposizione dello schema OpenAPI generato tramite un'interfaccia utente visiva o un file serializzato.

Esercitazione7 – Web API

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione7
  - `dotnet new webapi -n demoApi8 -f` **`net8.0`**
- Analizziamo

Esercitazione7 – Web API

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
if (app.Environment.IsDevelopment()){
    app.UseSwagger();
    app.UseSwaggerUI();}
app.UseHttpsRedirection();
var summaries = new[]{"Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm", "Balmy", "Hot",
"Sweltering", "Scorching"};
app.MapGet("/weatherforecast", () =>{
    var forecast =  Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast(
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)])).ToArray();
    return forecast;})
.WithName("GetWeatherForecast")
.WithOpenApi();
```

Program.cs

Le API minime offrono il supporto predefinito per la generazione di informazioni sugli endpoint in un'app tramite il pacchetto Microsoft.AspNetCore.OpenApi.

- AddEndpointsApiExplorer : configura l'app per l'uso di Esplora API per individuare e descrivere gli endpoint con annotazioni predefinite.
- WithOpenApi esegue l'override delle annotazioni predefinite generate da Esplora API con quelle generate dal pacchetto Microsoft.AspNetCore.OpenApi.
- UseSwagger aggiunge il middleware Swagger.
- 'UseSwaggerUI' abilita una versione incorporata dello strumento dell'interfaccia utente di Swagger.
- WithNameIEndpointNameMetadata: nell'endpoint viene usato per la generazione di collegamenti e viene considerato come l'ID operazione nella specifica OpenAPI dell'endpoint specificato.

Esercitazione7 – Web API

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi"
Version="8.0.8" />
    <PackageReference Include="Swashbuckle.AspNetCore"
Version="6.4.0" />
  </ItemGroup>

</Project>
```

Esercitazione7.csproj
Microsoft.AspNetCore.OpenApi viene aggiunto come PackageReference a un file di progetto.

Esercitazione7 – Web API

GET, POST, PUT e DELETE nella progettazione di API RESTful

Metodi HTTP:

- GET : Utilizzato per recuperare dati da un server.
- POST : Utilizzato per inviare dati al server per creare una nuova risorsa.
- PUT : utilizzato per aggiornare o sostituire una risorsa esistente.
- DELETE : Utilizzato per rimuovere una risorsa dal server.

Esercitazione7 – Web API

```
app.MapGet("/weatherforecast", () =>
{
    var forecast =  Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast")
.WithOpenApi();
```

Program.cs – Minimal API

Le API minime sono un approccio semplificato per la creazione di API HTTP veloci con ASP.NET Core. È possibile creare endpoint REST completamente funzionanti con codice e configurazione minimi. Salta l'impalcatura tradizionale ed evita controller non necessari dichiarando in modo fluido percorsi e azioni API.

Un API endpoint è un luogo digitale esposto tramite l'API dal quale l'API riceve le richieste e invia le risposte. Ogni endpoint è un URL (Uniform Resource Locator) che fornisce la posizione di una risorsa sul server dell'API.

Un'API è un insieme di protocolli e strumenti per facilitare l'interazione tra due applicazioni. Un endpoint è un posto sull'API dove avviene lo scambio. Gli endpoint sono URI (Uniform Resource Identifiers) su un'API a cui un'applicazione può accedere.
All'endpoint, l'API accederà alle risorse di cui ha bisogno da un server per eseguire un'attività specifica, come il recupero di determinati dati o informazioni.

Esercitazione7 – Web API



Esercitazione7 – Web API (.NET 8)

- Compilare e Consumare Web API
  - `dotnet build`
  - `dotnet run`
  - `…e poi???`
  - `vedi slide successiva >>>`
- Controllare la configurazione dell'applicazione (`launchSettings.json`)

Esercitazione7 – Web API

Avviare il progetto

Nel terminale apparirà un indirizzo per testare la web api con Swagger:

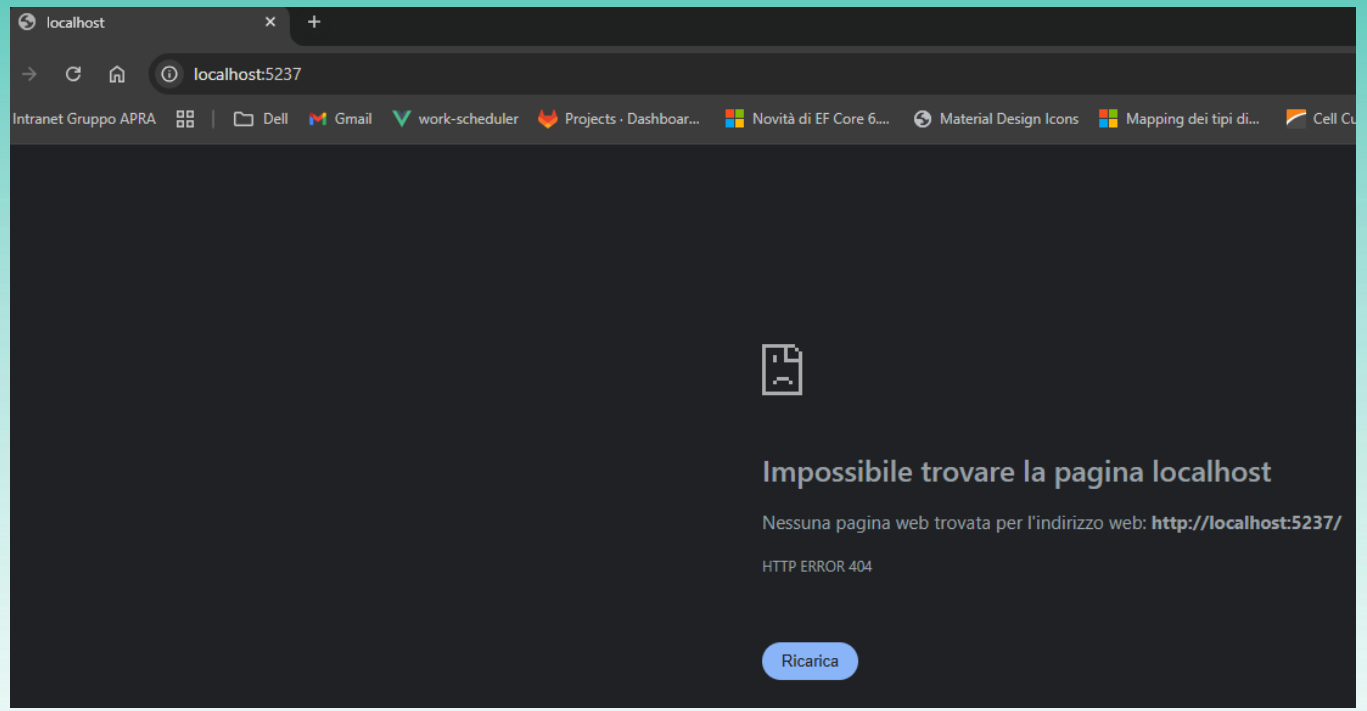
```
PS C:\Users\b.pierangeli\Desktop\CorsoITS\Esercitazione7> dotnet run
Compilazione...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5237
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\b.pierangeli\Desktop\CorsoITS\Esercitazione7
```

Posizionarsi sopra l'url e CTRL+click con il tasto sinistro del mouse.

Si aprirà una tab nel browser, come nella slides seguente

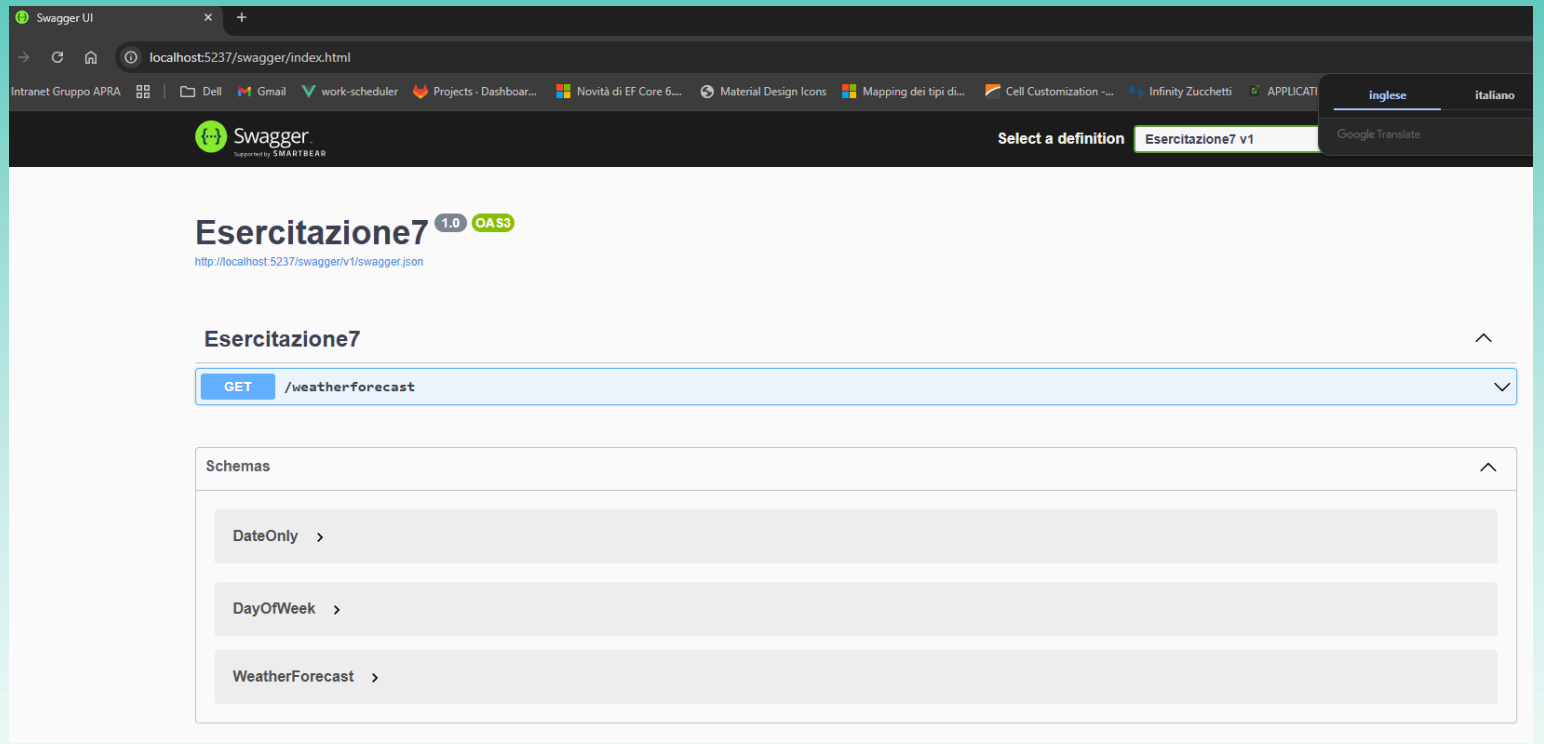
Per stoppare l'esecuzione del progetto API CTRL+C

Esercitazione7 – Web API



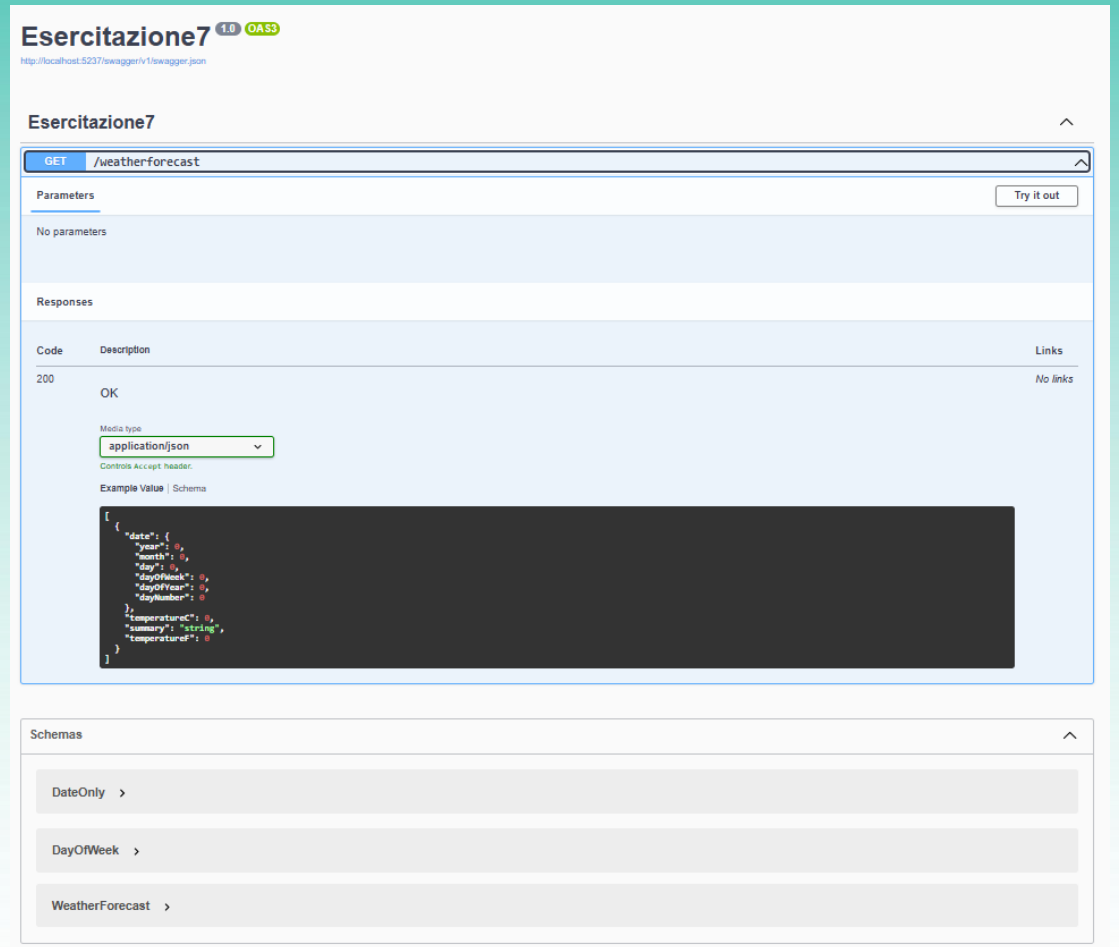
Aggiungere /swagger all'url e cliccare invio. Vedi slide successiva

Esercitazione7 – Web API



Quella che appare è l'interfaccia grafica di swagger, da cui possiamo testare le api del progetto.

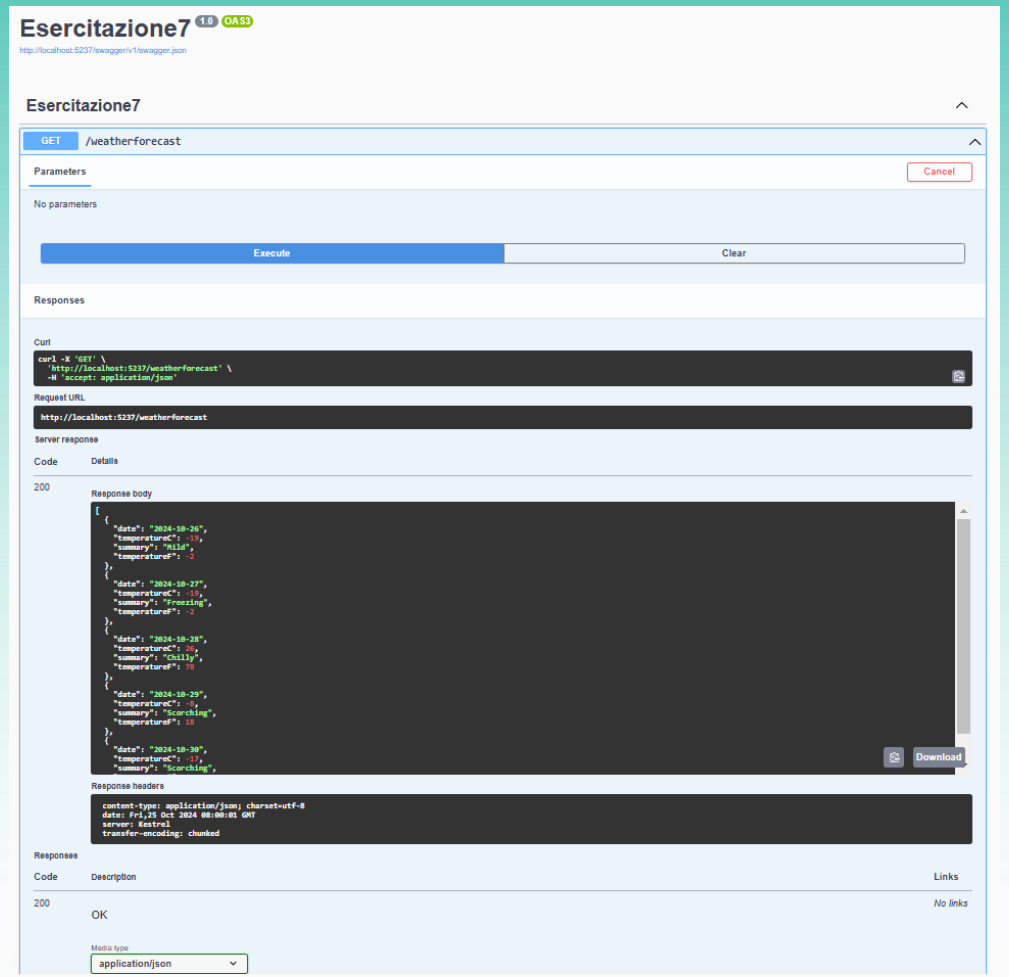
Esercitazione7 – Web API



Vediamo:

1. Il nome del progetto (Esercitazione7);
2. La chiamata HTTP GET e l'endpoint (/weatherforecast);
3. Parameters, se presenti (in questo caso No parameters);
4. Responses, sono le risposte accettate, in questo caso:
   - Status CODE: 200
   - Tipo della risposta: application/json.

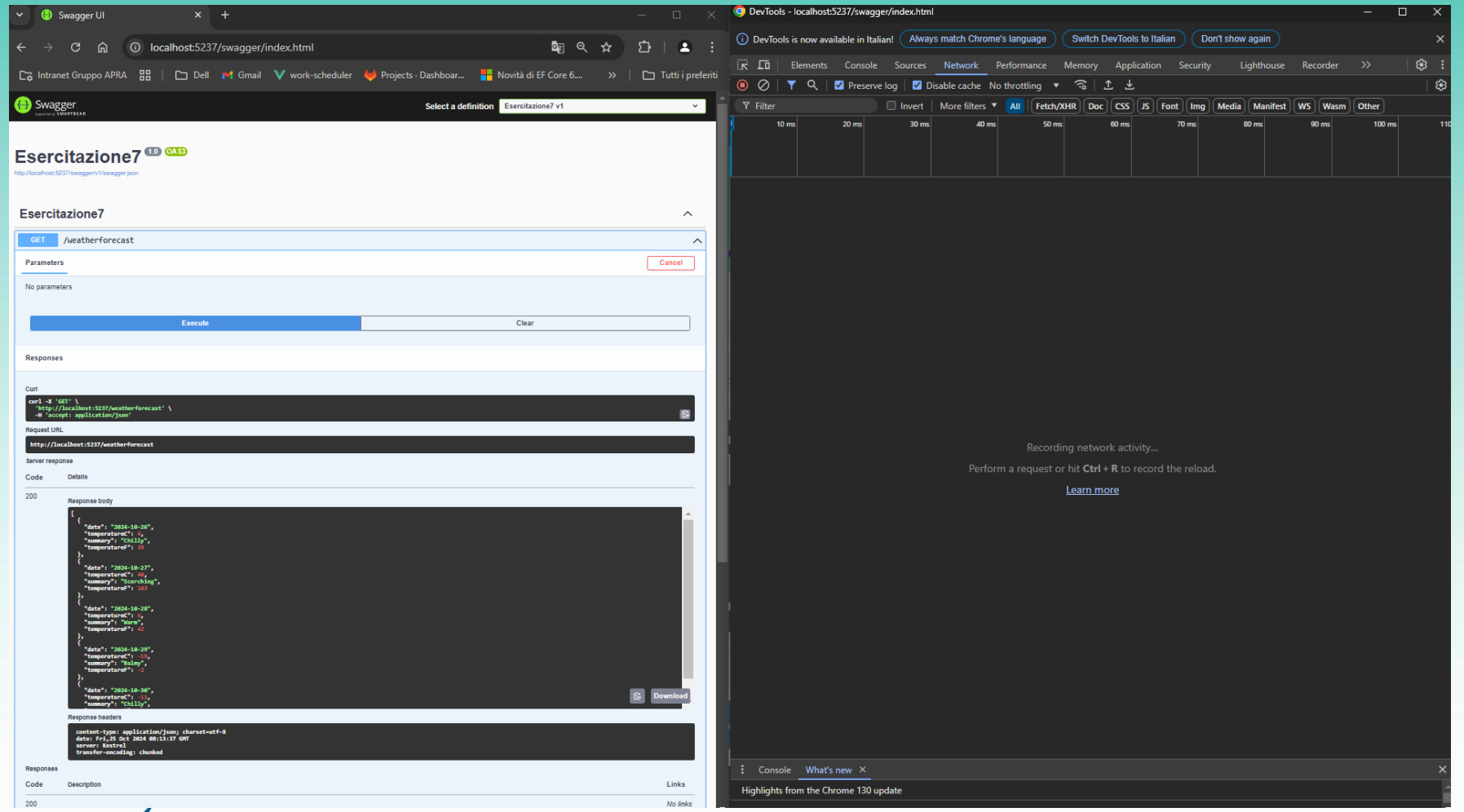
Esercitazione7 – Web API



Per eseguire, cliccare su Try it out e Execute.
Otteniamo la risposta (Repsponse body).

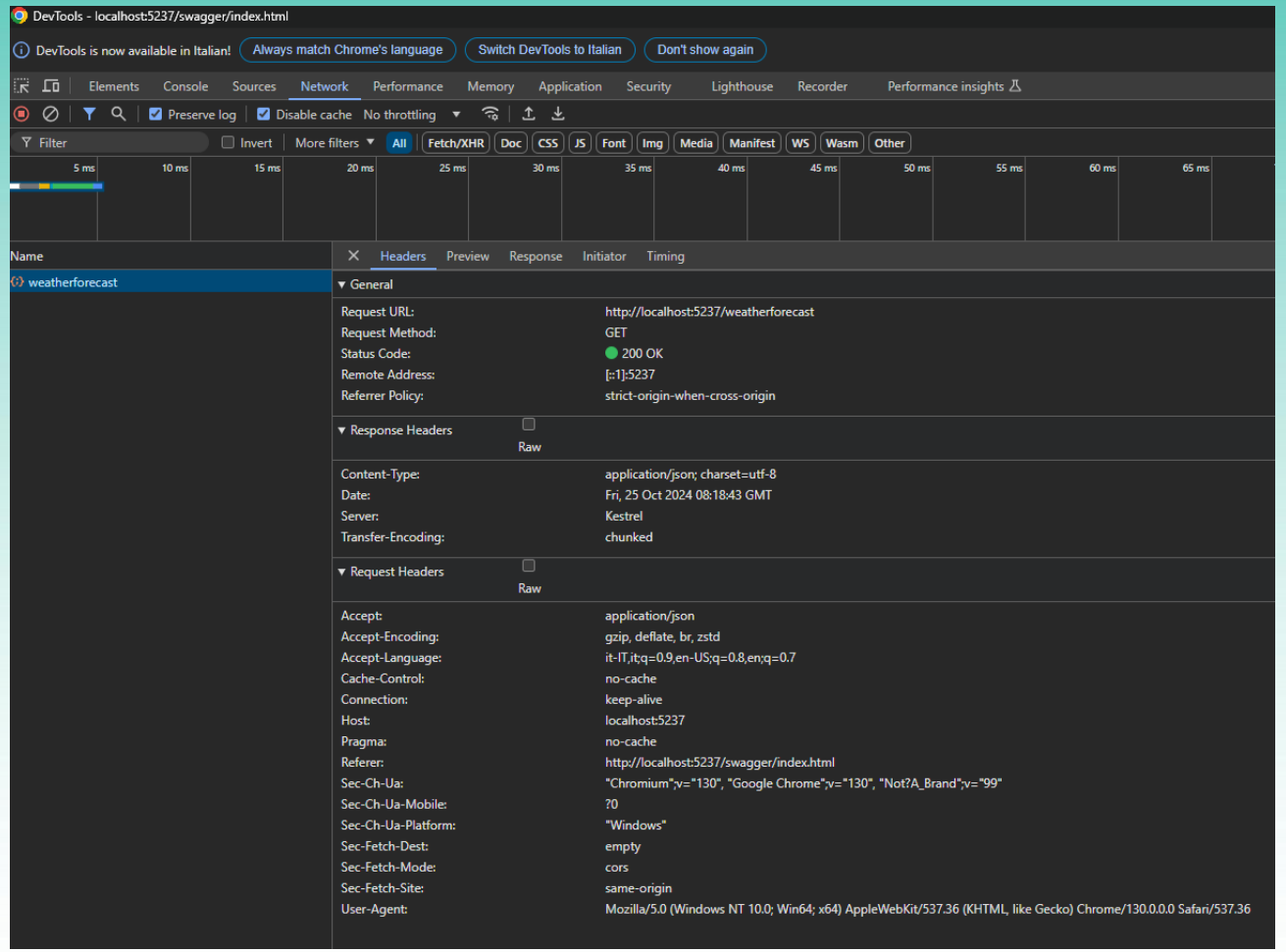
Ora clicchiamo F12 per aprire gli strumenti dello sviluppatore e posizioniamoci nella tab Network, come nella slide successiva

Esercitazione7 – Web API



Se da qui ora clicchiamo su Execute, vediamo che la tab Network si popola con una riga, se clicchiamo in quella riga vediamo tutte le informazioni relative alla chiamata HTTP, come nella slide successiva

Esercitazione7 – Web API



Nella tab:

- Headers, ritroviamo le informazioni generali, con i dettagli della chiamata: l'URL(Request URL), il tipo di chiamata(Request Method), lo status code della risposta e il tipo di risposta accettata(Accept);
- Preview o Response, ritroviamo i dati inviati dal server come risposta, se presenti;
- Se avessimo avuto dei parametri nel body avremmo visualizzato anche una ulteriore tab (Payload) con i dati inviati dal client al server.

Esercitazione7 – Web API (.NET 9)

- Creare un nuovo progetto, come prima ma su .NET 9
  - `dotnet new webapi -n demoApi9 -f net9.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede? https://medium.com/@dev.mjdhanesh/scalar-api-reference-in-net-9-a-modern-alternative-to-swagger-cf2d48cdd258

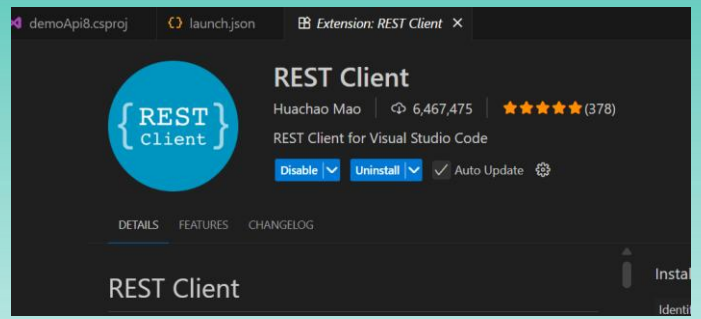
Esercitazione7 – Web API (.NET 9)

- Creare un nuovo progetto, come prima ma su .NET 9
  - `dotnet new webapi -n demoApi9 -f net9.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede? https://medium.com/@dev.mjdhanesh/scalar-api-reference-in-net-9-a-modern-alternative-to-swagger-cf2d48cdd258
  - Aggiungere e configurare Scalar
- Utilizzare un client REST esterno:
  - REST client (VS code ext)
  - Postman,
  - Bruno

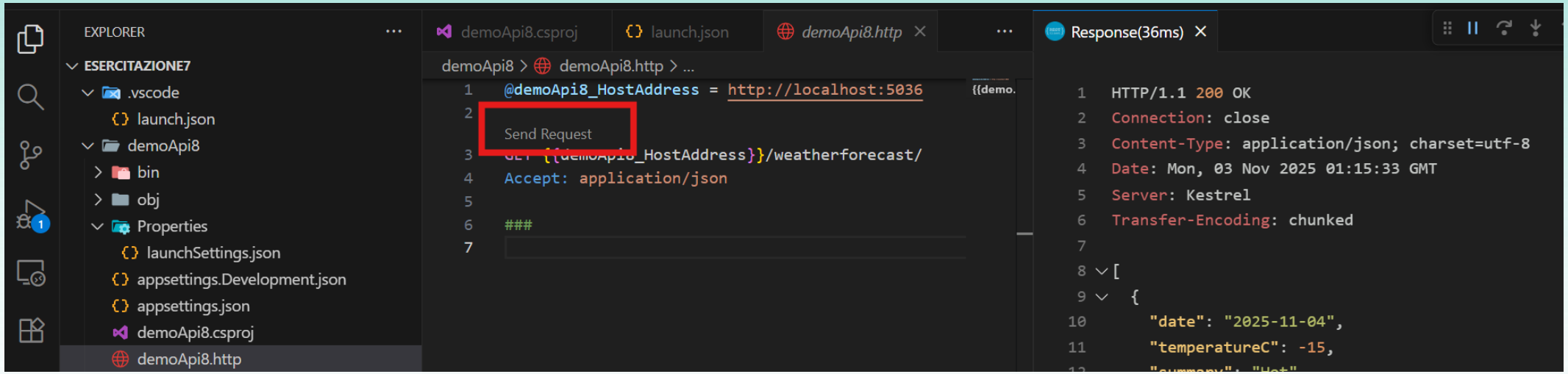




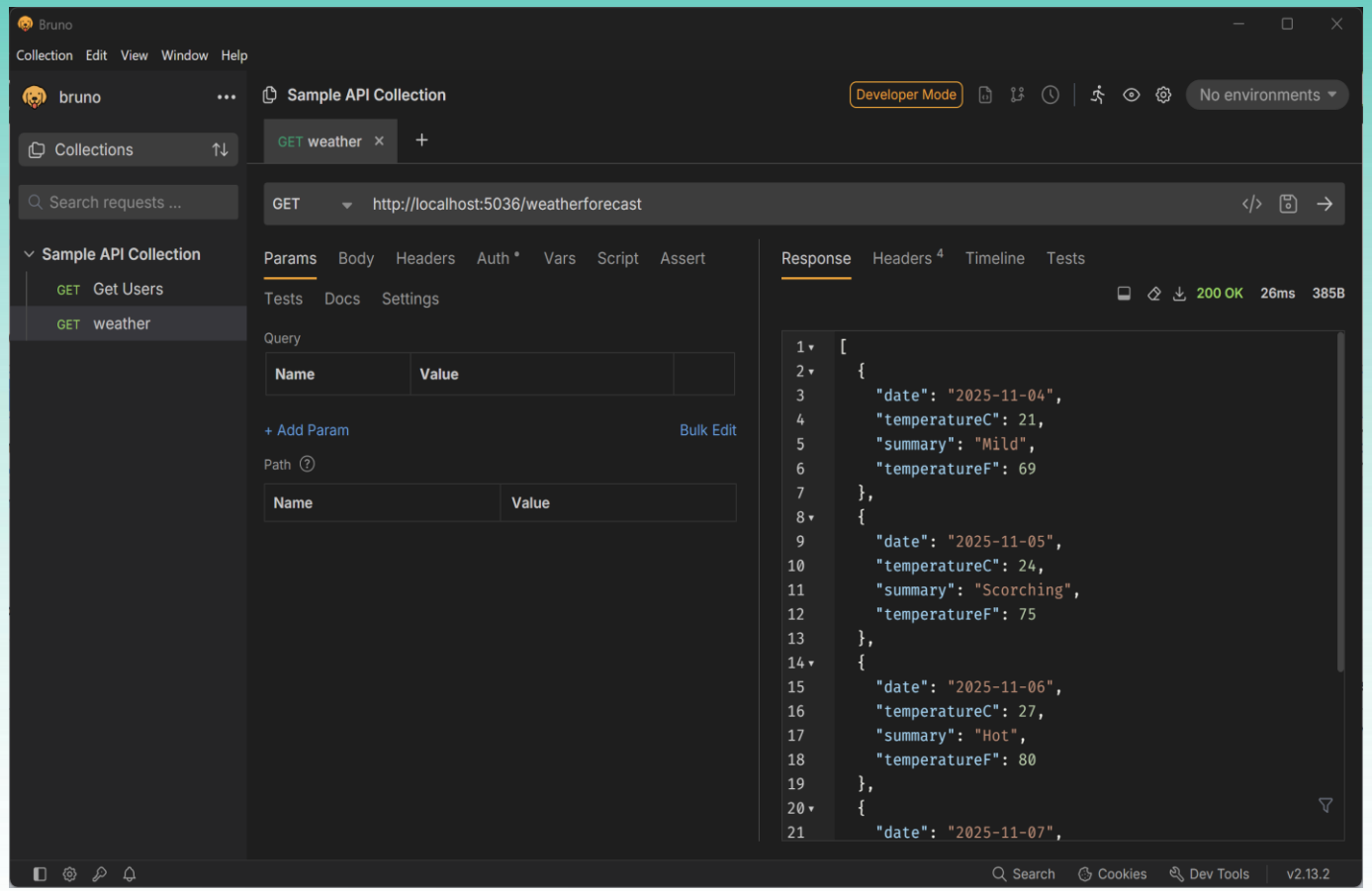
Esercitazione7 – Web API (REST Client)



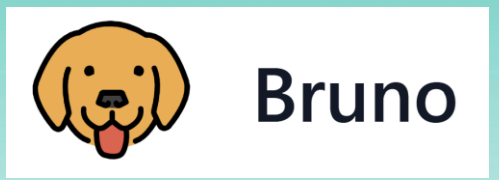
- Ha come vantaggio la possibilità di sfruttare al volo le richieste .http nella codebase
- Come testare?



Esercitazione7 – Web API (Bruno)



Perché Bruno e non Postman?



https://www.usebruno.com/compare/bruno-vs-postman

Bruno semplifica il test delle API grazie a un'interfaccia grafica, gestione di collezioni e variabili, e strumenti avanzati che migliorano la produttività rispetto a client REST più semplici.

ISTITUTI TECNICI SUPERIORI
IT'S MY FUTURE

Finanziato dall'Unione europea
NextGenerationEU

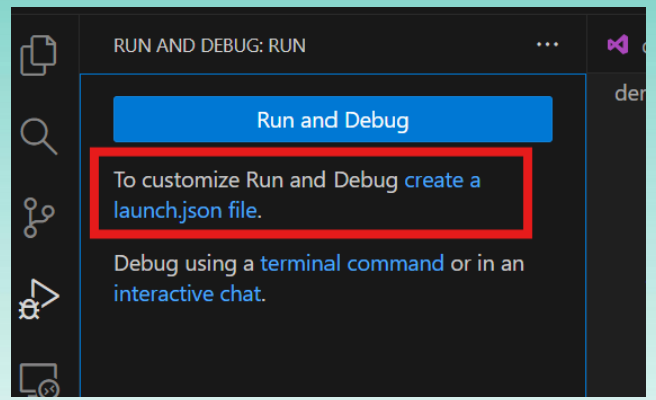
Ministero dell'Istruzione e del Merito

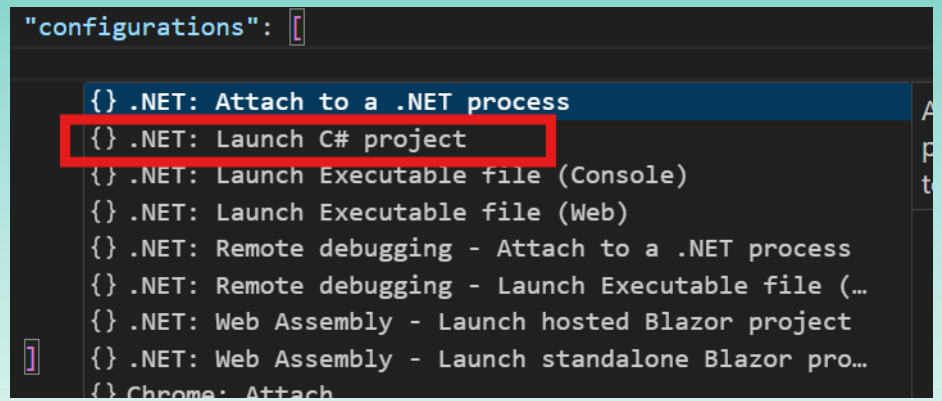
Italiadomani

its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione7 – Launcher VS Code

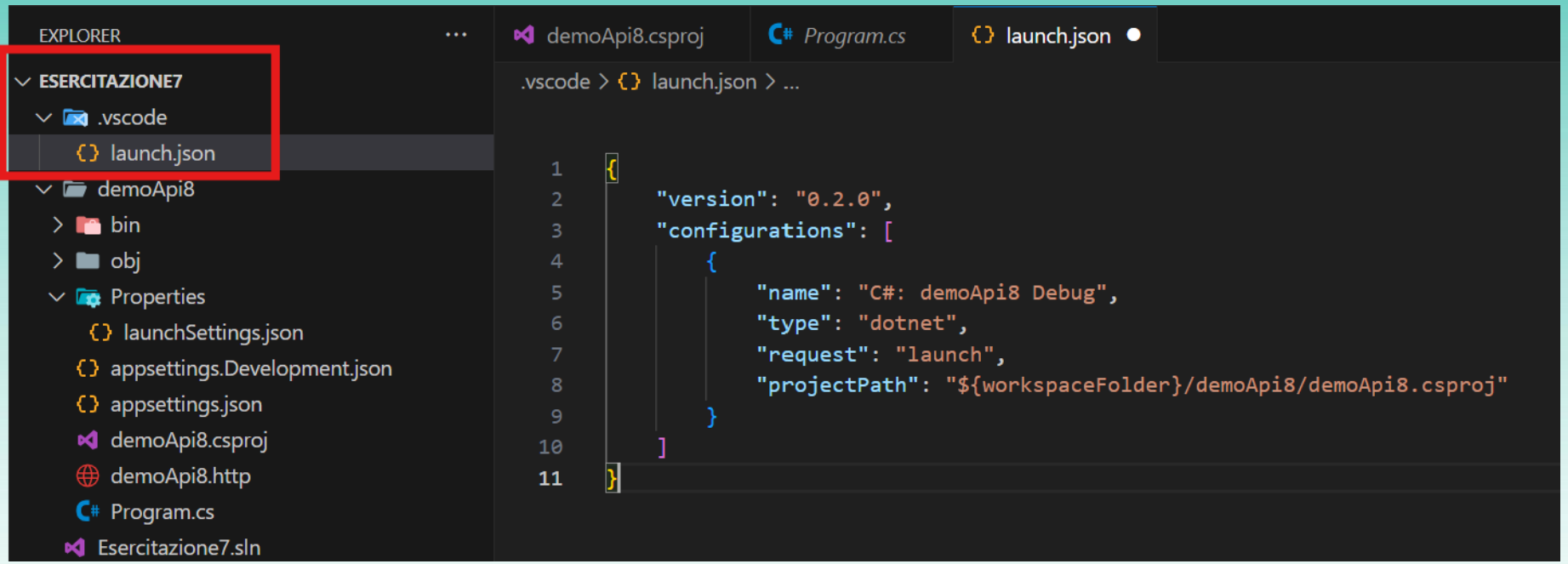
- Creare una configurazione per il debug di VS code (tipo debugger, progetto di avvio, gli argomenti, variabili di ambiente)
- Selezionare C#






```
// For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
"version": "0.2.0",
"configurations": [
    {
        "name": "C#: <project-name> Debug",
        "type": "dotnet",
        "request": "launch",
        "projectPath": "${workspaceFolder}/<relative-path-to-project-folder><project-name>.csproj"
    }
```

Esercitazione7 – Launcher VS Code



Provare da scorciatoia F5 (o FN+F5 a seconda dell'impostazione del bios)

Esercitazione7 – Launcher VS Code

E per lanciare entrambi i progetti insieme?

Esercitazione7 – Launcher VS Code

….provatelo a casa!!!

Suggerimento 👇
https://learn.microsoft.com/en-us/microsoft-edge/visual-studio-code/microsoft-edge-devtools-extension/launch-json

Esercitazione8 – Web API Person

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione8
  - `dotnet new webapi -n Esercitazione8`

Esercitazione8 – Web API Person

```
public class Person
{
    public int id { get; set; }
    public string? Name { get; set; }
    public int Age { get; set; }
}
```

Person.cs
Creiamo una classe Person come in figura

Esercitazione8 – Web API Person

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
```

Program.cs
Personalizziamo ora il nostro Program, sempre utilizzando l'interfaccia grafica di Swagger, ma utilizzando le nostre API.

A fianco vediamo la prima parte del codice, essenzialmente uguale al Program.cs che avevamo nella precedente Esercitazione.

Esercitazione8 – Web API Person

```
Dictionary<int, Person> dict = new Dictionary<int, Person>();
app.MapGet("/getPeople", () =>
{
    if(dict.Count > 0)
        return dict.Values.ToArray();
    else
        return [];
});
app.MapGet("/getPerson/{id}", (int id) =>
{
    if(dict.Count > 0 && dict.ContainsKey(id))
        return dict[id];
    else
        return null;
});
```

Program.cs

Utilizziamo un Dictionary di tipo chiave-valore per simulare una tabella del DB (la chiave corrisponde all'id della persona, i valori a Person), ogni elemento di dict corrisponde a una persona con chiave univoca id.

Inseriamo poi le API:

- getPeople chiamata GET senza parametri che controlla se dict contiene degli elementi, in tal caso restituisce un array di tutte i valori di dict (tutte le Person sotto forma di array), altrimenti restituisce un array vuoto;
- getPerson/id chiamata GET con parametro id (nell'url) che controlla che dict contiene degli elementi e che contiene la chiave id, in tal caso restituisce un valore Person di dict con chiave uguale a id (dict[id]), altrimenti restituisce un array null;

Esercitazione8 – Web API Person

```
app.MapPost("/create", (Person person) =>
{
    if (dict.Count > 0 && dict.ContainsKey(person.id))
        return Results.BadRequest();
    else
    {
        dict.Add(person.id, person);
        return Results.Ok();
    }
});
app.MapPut("/update", (Person person) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict[person.id] = person;
        return Results.Ok();
    }
});
```

Program.cs

- Create chiamata POST con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id del parametro person è presente come chiave in dict, in tal caso restituisce BadRequest, altrimenti inserisce un nuovo elemento in dict (con chiave person.id e valore person) e restituisce OK.
- Update chiamata PUT con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id di person non è presente come chiave in dict oppure controlla che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti modifica l'elemento di dict con chiave uguale a person.id con i nuovi valori del parametro person e restituisce OK;

Esercitazione8 – Web API Person

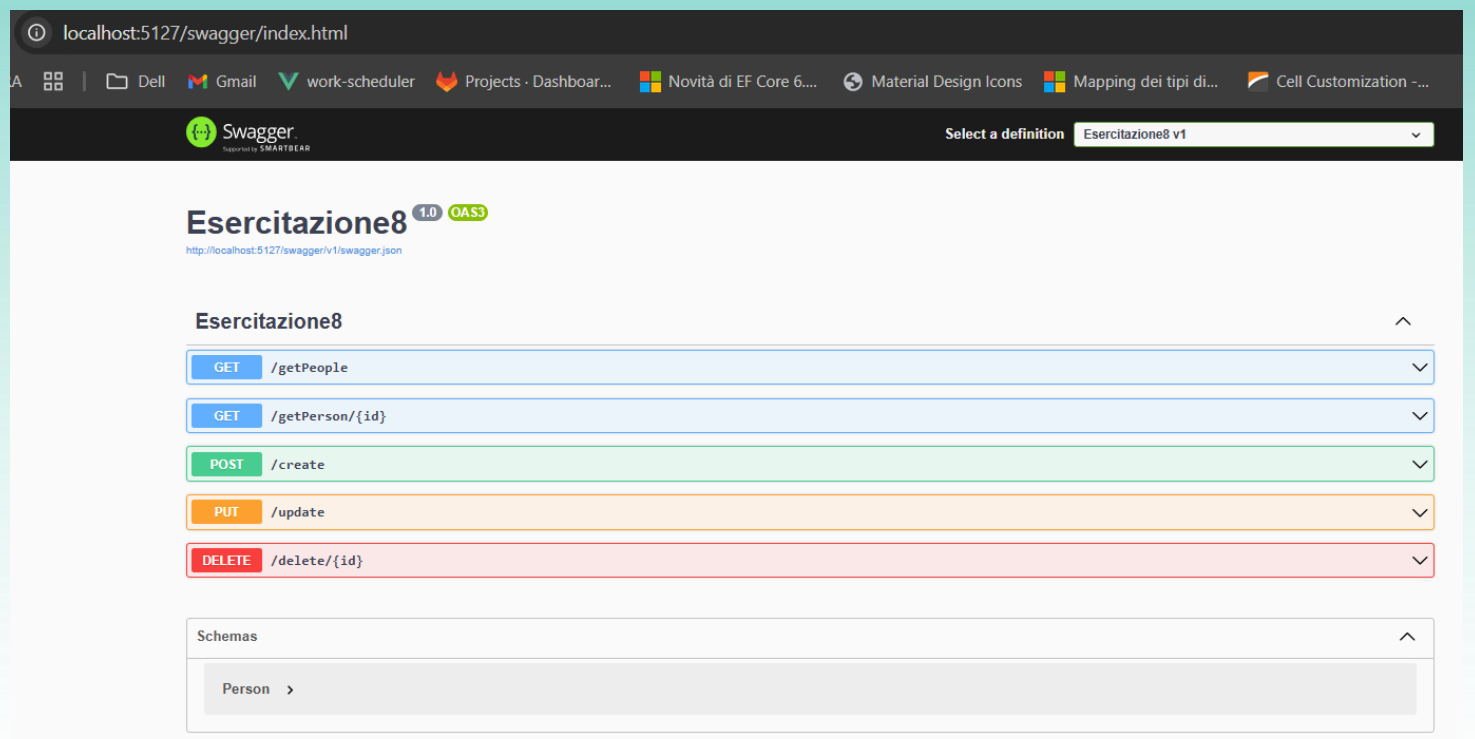
```
app.MapDelete("/delete/{id}", (int id) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(id)) ||
dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict.Remove(id);
        return Results.Ok();
    }
});
app.Run();
```

Program.cs

- Delete/id chiamata DELETE con parametro id (nell'url) che controlla se dict contiene elementi e che l'id non è presente come chiave in dict oppure che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti elimina l'elemento di dict con chiave id e restituisce OK.

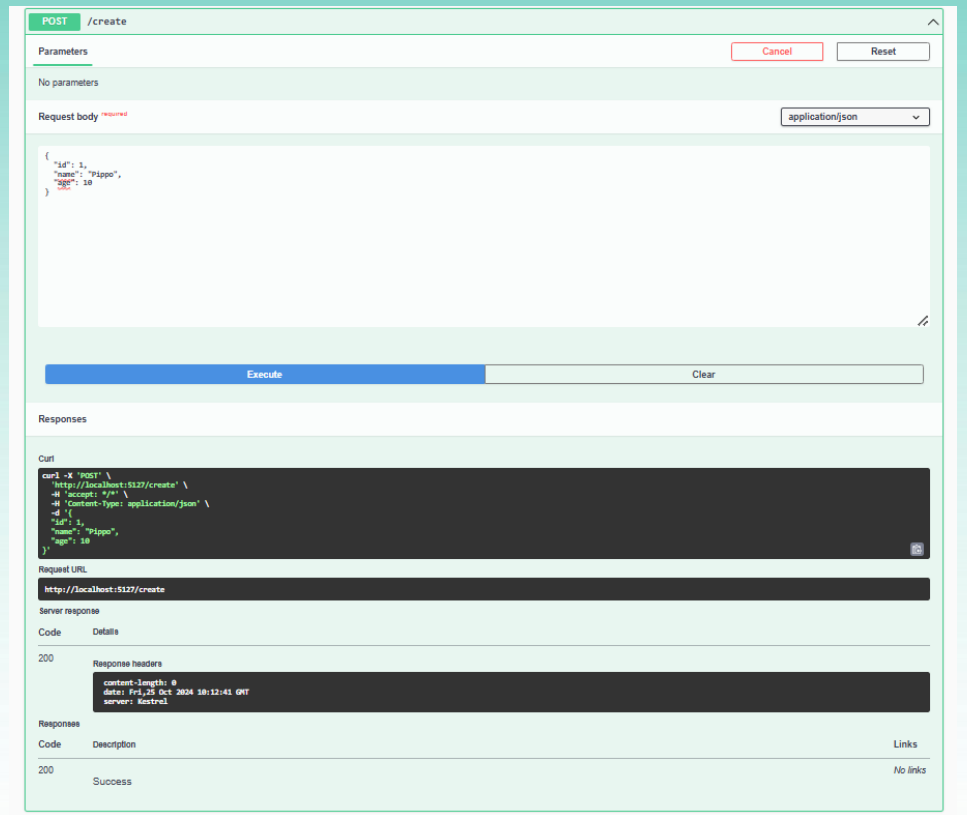
Esercitazione8 – Web API Person

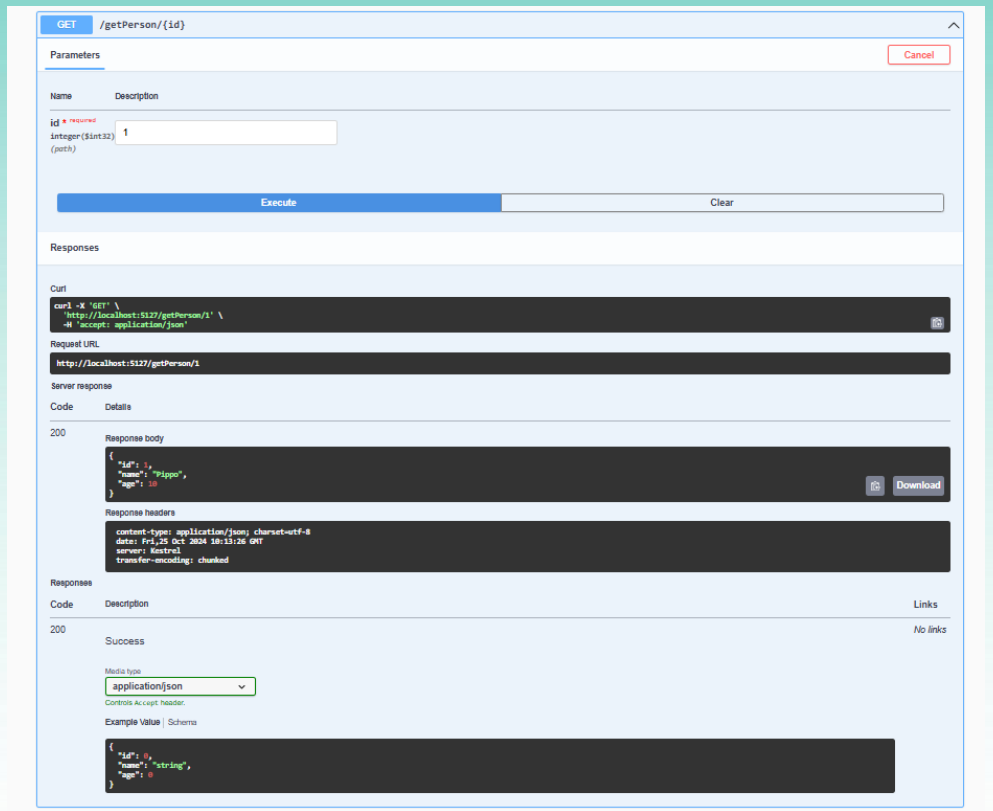
Avviando l'app da terminale con il comando
dotnet run
e aprendo Swagger nel browser possiamo testare le api:



Esercitazione8 – Web API Person

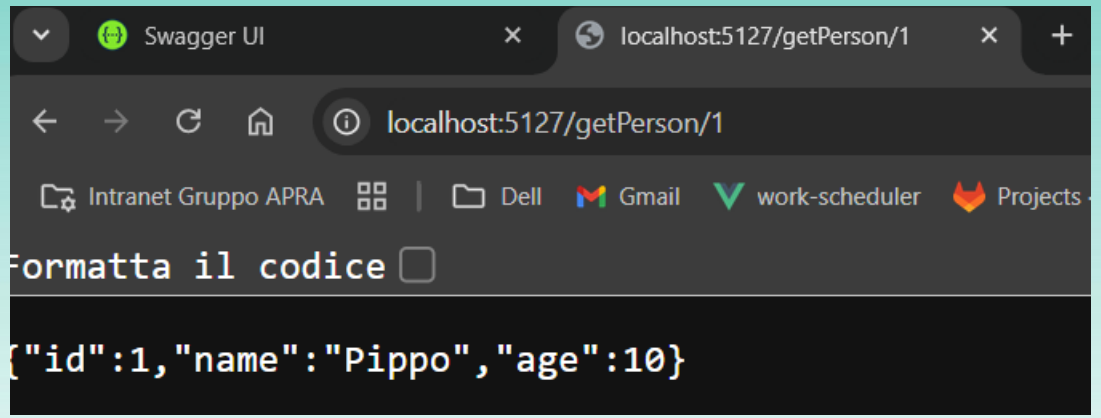
Esempio di inserimento di una persona e successiva lettura tramite id (notare che la POST passa un json come parametro, la GET passa il parametro nell'url):





Esercitazione8 – Web API Person

E’ possibile fare la stessa chiamata precedente anche in una nuova scheda del browser come in seguito:



Quindi non ci stiamo interfacciando con Swagger, ma stiamo richiamando direttamente l’endpoint getPerson con parametro id = 1

Esercitazione9 – CSR

Cerchiamo di dare un’architettura pulita alla nostra applicazione.

Quindi partendo dall’Esercitazione8, la nuova applicazione avrà le stesse funzionalità precedenti, ma la struttura del progetto verrà modificata in modo da rispettare la CLEAN ARCHITECTURE.

Oggi nel mondo del software è molto diffuso l’uso delle cosiddette Clean Architecture. Vengono chiamate così perché si basano tutte sullo stesso principio di progettazione del software: la separazione delle responsabilità.

Esercitazione9 – CSR

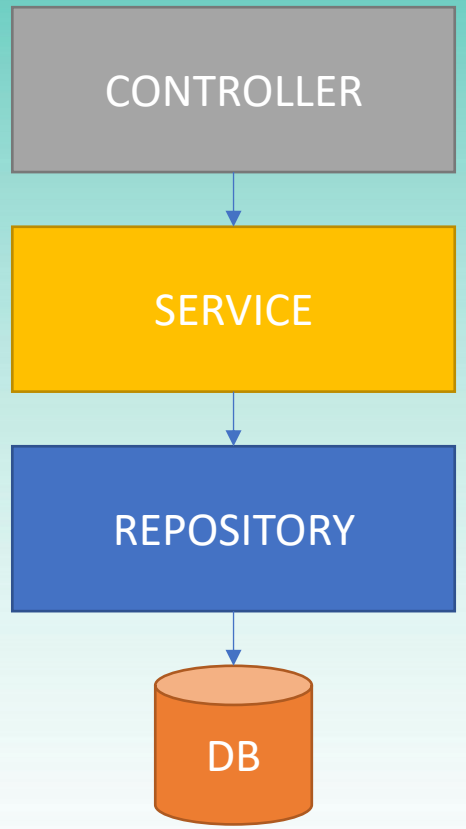
MODELLO CSR

Nello sviluppo software, creare applicazioni scalabili, manutenibili e testabili è fondamentale.

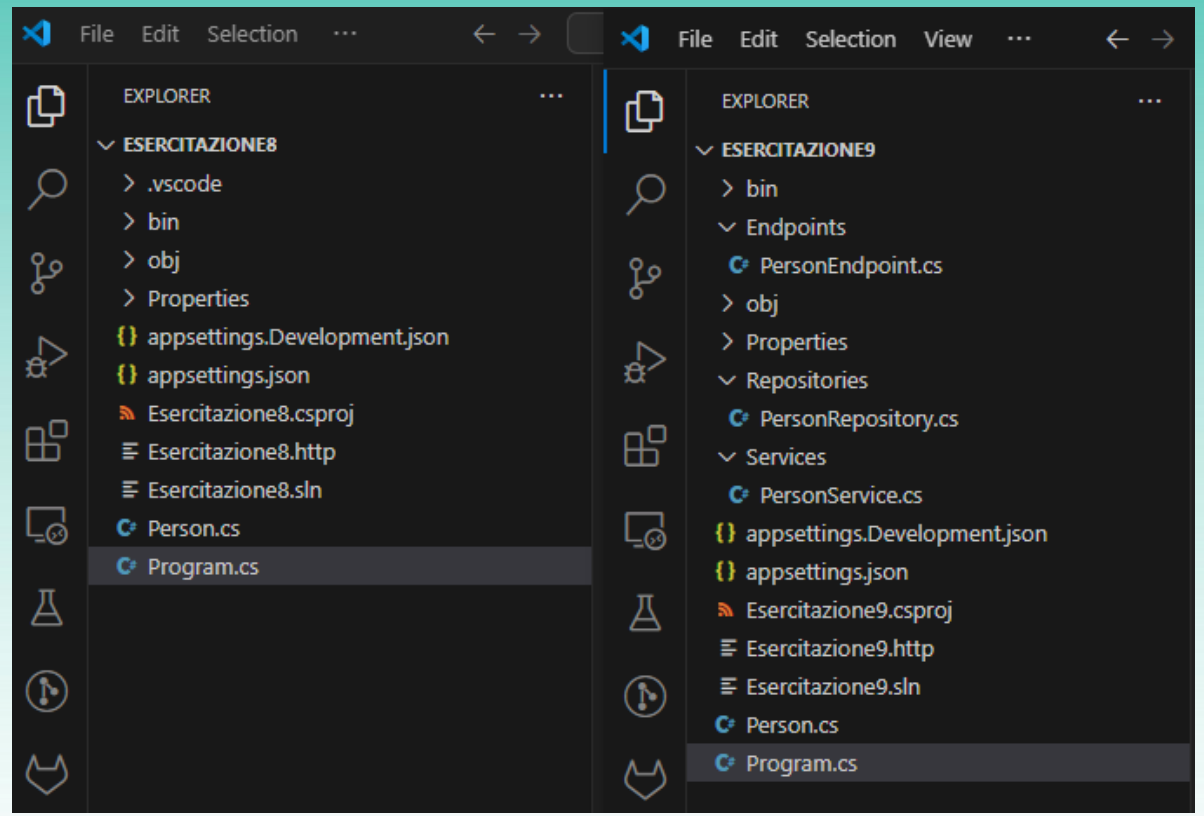
Un modello architettonico che aiuta a raggiungere questi obiettivi è il modello Controller-Service-Repository (CSR). Questo modello promuove la separazione delle preoccupazioni dividendo la logica dell'applicazione in tre livelli distinti: il Controller, il Service e il Repository.

Ogni livello ha una responsabilità specifica, rendendo il codice più facile da gestire, testare e scalare:

- il livello Controller gestisce le richieste HTTP in arrivo, le elabora e restituisce le risposte HTTP appropriate. Agisce come intermediario tra il client e il livello Service;
- il livello Service contiene la logica aziendale dell'applicazione. Elabora i dati ricevuti dal controller, esegue le operazioni necessarie e comunica con il livello di Repository per recuperare o rendere persistenti i dati;
- il livello Repository, gestisce l'accesso ai dati e la persistenza. Comunica con il database o altre fonti di dati, incapsulando la logica per interrogare, salvare e aggiornare i dati.



Esercitazione9 – CSR



Differenze fra Esercitazione8 e Esercitazione9

Mettiamo a confronto l'architettura delle due applicazioni:

- Nell'Esercitazione8 avevamo creato una solo classe Person all'interno del file Person.cs e endpoint, logica dell'applicazione e accesso ai dati avvenivano all'interno del Program.cs;
- Nell'Esercitazione9 sono invece presenti altre cartelle che permettono di rispettare la separazione delle responsabilità:
    1. Il livello Controller lo troviamo all'interno del file PersonEndpoint.cs della cartella Endpoints;
    2. Il livello Service lo troviamo all'interno del file PersonService.cs della cartella Services;
    3. Il livello Repository lo troviamo all'interno del file PersonRepository.cs della cartella Repositories.

Le cartelle Endpoints, Services e Repositories ci permettono di suddividere le responsabilità e di farlo per ogni singolo modello (classe).
Quindi, se oltre alla classe Person di Person.cs in futuro aggiungeremo un'altra classe (ad esempio la classe Animal in un file Animal.cs), aggiungeremo altri file nelle cartelle Endpoints, Services e Repositories: esattamente un ulteriore file per ogni livello di responsabilità, cioè per ogni cartella (rispettivamente i files AnimalEndpoint.cs, AnimalService.cs e AnimalRepository.cs).

Esercitazione9 – CSR

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

app.MapAPIEndpoints();

// Configure the HTTP request pipeline.

if (app.Environment.IsDevelopment())

{

    app.UseSwagger();

    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.Run();
```

Program.cs

Rispetto all'Esercitazione8, nel Program.cs, togliamo tutto il codice che appartiene ai livelli di responsabilità CSR, per spostarli in altri files.
Ricordiamo però che il Program.cs è il punto di avvio dell'applicazione ed è quello che si occupa di creare ed eseguire l'app.
E' quindi necessario dire all'app di aggiungere le nostre API, altrimenti non verrano create.
Per farlo richiamiamo un metodo, statico, creato da noi: MapAPIEndpoints() che contiene la definizione delle API.
Vediamo con la slide successiva la definizione del metodo MapAPIEndpoints() della classe PersonEndpoints.

Esercitazione9 – CSR

```
public static class PersonEndpoints

{

    private static PersonService personService = new
PersonService();

    public static void MapAPIEndpoints(this
IEndpointRouteBuilder app)

    {

        app.MapGet("/getPeople", () =>
personService.GetPeople());

        app.MapGet("/getPerson/{id}", (int id) =>
personService.GetPerson(id));

        app.MapPost("/create", (Person person) =>

        {

            bool result =
personService.Create(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.BadRequest();

        });
```

```
        app.MapPut("/update", (Person person) =>

        {

            bool result =
personService.Update(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

        app.MapDelete("/delete/{id}", (int id) =>

        {

            bool result = personService.Delete(id);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

    }

}
```

PersonEndpoints.cs
Il file contiene la definizione di una classe statica: PersonEndpoints.

Questa classe è quella che deve contenere il livello Controller e quindi la definizione delle API, ma non deve contenere le responsabilità del livello Service e del livello Repository.

Ricordiamo che una classe statica non può essere istanziata e può contenere solo membri statici.

La classe PersonEndpoints ha:

- Un istanza statica di PersonService, che come vedremo è una classe che contiene il livello Service;
- Il metodo statico MapAPIEndpoints, che contiene la definizione delle API e le aggiunge all'app. Dentro le API non troviamo logica. Ogni API utilizza il membro statico personService della classe PersonEndpoints per richiamare i metodi della classe PersonService.

Ogni API della classe PersonEndpoints quindi richiama metodi del Service, prende la risposta e restituisce una risposta al chiamante (client).

Vediamo con la slide successiva la classe PersonService.

Esercitazione9 – CSR

```
public class PersonService

{

    private PersonRepository personRepository = new
PersonRepository();

    public IEnumerable<Person> GetPeople()

    {

        return personRepository.GetPeople();

    }

    public Person? GetPerson(int id)

    {

        return personRepository.GetPerson(id);

    }

    public bool Create(Person person)

    {

        return personRepository.Create(person);

    }

    public bool Update(Person person)

    {

        return personRepository.Update(person);

    }

    public bool Delete(int id)

    {

        return personRepository.Delete(id);

    }

}
```

PersonService.cs
Il file contiene la definizione di una classe: PersonService.

Questa classe è quella che deve contenere il livello Service e quindi la logica aziendale dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Repository.

La classe PersonService ha:

- Un istanza di PersonRepository, che come vedremo è una classe che contiene il livello Repository;
- La definizione dei metodi richiamati nelle API della classe PersonEndpoints. Questi metodi attualmente non contengono logica, nel nostro caso non stiamo manipolando i dati che ci arrivano in ritorno dal livello Repository, ma avremmo dovuto farlo qui, se necessario. Dentro i metodi non effettuiamo l'accesso diretto ai dati, perché ciò aspetta al livello Repository, infatti ogni metodo della classe PersonService utilizza il membro personRepository della classe PersonService per richiamare i metodi della classe PersonRepository.

Ogni metodo della classe PersonService quindi richiama metodi del Repository, prende la risposta e, dopo aver inserito la logica dell'applicativo, se presente, restituisce una risposta al chiamante (API del livello Controller).

Vediamo con la slide successiva la classe PersonRepository.

Esercitazione9 – CSR

```
public class PersonRepository

{

    private Dictionary<int, Person> dict = new Dictionary<int, Person>();

    public IEnumerable<Person> GetPeople()

    {

        if(dict.Count > 0)

            return dict.Values.ToArray();

        else

            return [];

    }

    public Person? GetPerson(int id)

    {

        if(dict.Count > 0 && dict.ContainsKey(id))

            return dict[id];

        else

            return null;

    }

    public bool Create(Person person)

    {

        if (dict.Count > 0 && dict.ContainsKey(person.id))

            return false;

        else

        {

            dict.Add(person.id, person);

            return true;

        }

    }
```

```
    public bool Update(Person person)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)

            return false;

        else

        {

            dict[person.id] = person;

            return true;

        }

    }

    public bool Delete(int id)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(id)) || dict.Count == 0)

            return false;

        else

        {

            dict.Remove(id);

            return true;

        }

    }

}
```

PersonRepository.cs
Il file contiene la definizione di una classe: PersonRepository.

Questa classe è quella che deve contenere il livello Repository e quindi l'accesso ai dati dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Service (già definito in PersnService).

La classe PersonRepository ha:

- Un istanza di un Dictionary, che simula il nostro DB;
- La definizione dei metodi richiamati nel nella classe PersonService. Questi metodi si occupano di accedere, leggere e scrivere nei dati, che nel nostro caso sono rappresentati da dict. Ogni metodo della classe PersonRepository quindi prende i dati dal DB e li restituisce ai Controller, dopo aver inserito la logica dell'applicativo.

Ogni metodo della classe PersonRepository quindi si occupa di leggere e scrivere nei dati (nel nostro caso dict), e restituisce una risposta al chiamante (metodi del livello Service).

Risoluzione errori e launch progetti multipli

launch.json ● Program.cs

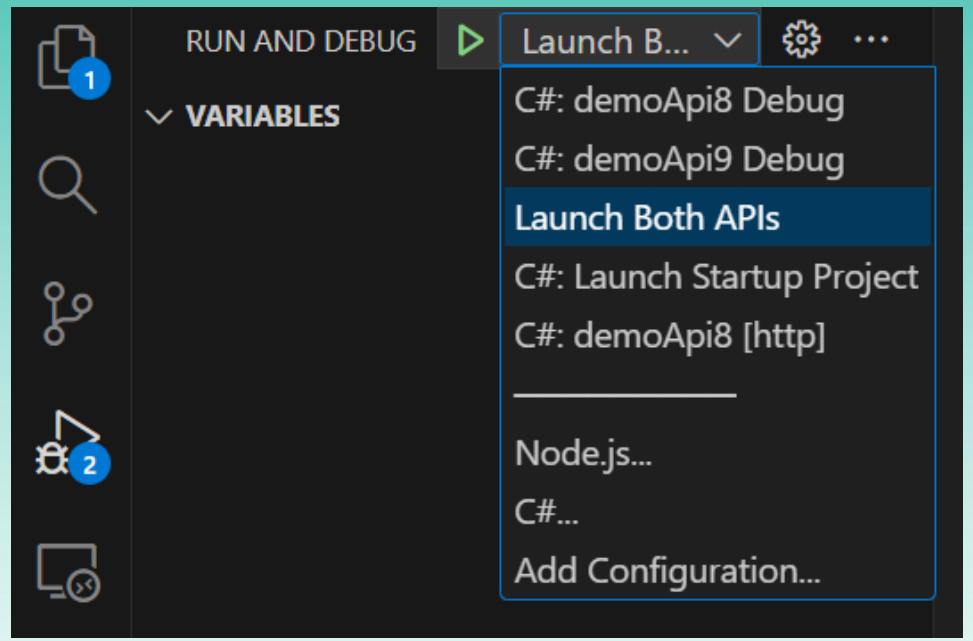
.vscode > launch.json > ...

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "C#: demoApi8 Debug",
            "type": "dotnet",
            "request": "launch",
            "projectPath": "${workspaceFolder}/demoApi8/demoApi8.csproj"
        },
        {
            "name": "C#: demoApi9 Debug",
            "type": "dotnet",
            "request": "launch",
            "projectPath": "${workspaceFolder}/demoApi9/demoApi9.csproj"
        }
    ],
    "compounds": [
        {
            "name": "Launch Both APIs",
            "configurations": [
                "C#: demoApi8 Debug",
                "C#: demoApi9 Debug"
            ],
            "stopAll": true
        }
    ]
}
```

https://learn.microsoft.com/en-us/microsoft-edge/visual-studio-code/microsoft-edge-devtools-extension/launch-json

- Aggiungere la configurazione del 2’ progetto all’array configurations[]
- Aggiungere sezione «compounds»: []
- stopAll: true, fermerà tutti i processi quando viene interrotto il debug

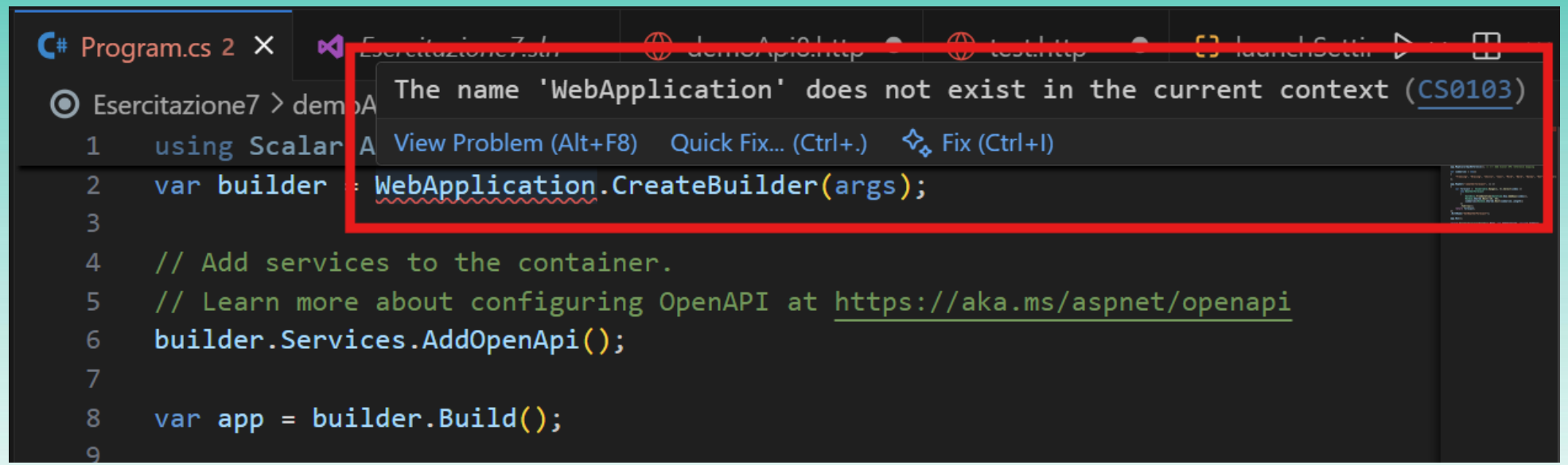
Risoluzione errori e launch progetti multipli



Nel pannello Debug, selezionando il menu sarà disponibile la nuova voce ‘Launch Both APIs’ come l’avevamo chiamata nel file di configurazione.

Selezionare e lanciare da F5 o da pulsante Play accanto

Risoluzione launch progetti multipli ed errori



Per la risoluzione si rimanda alla slide successiva >>

Gestione della Soluzione in .NET

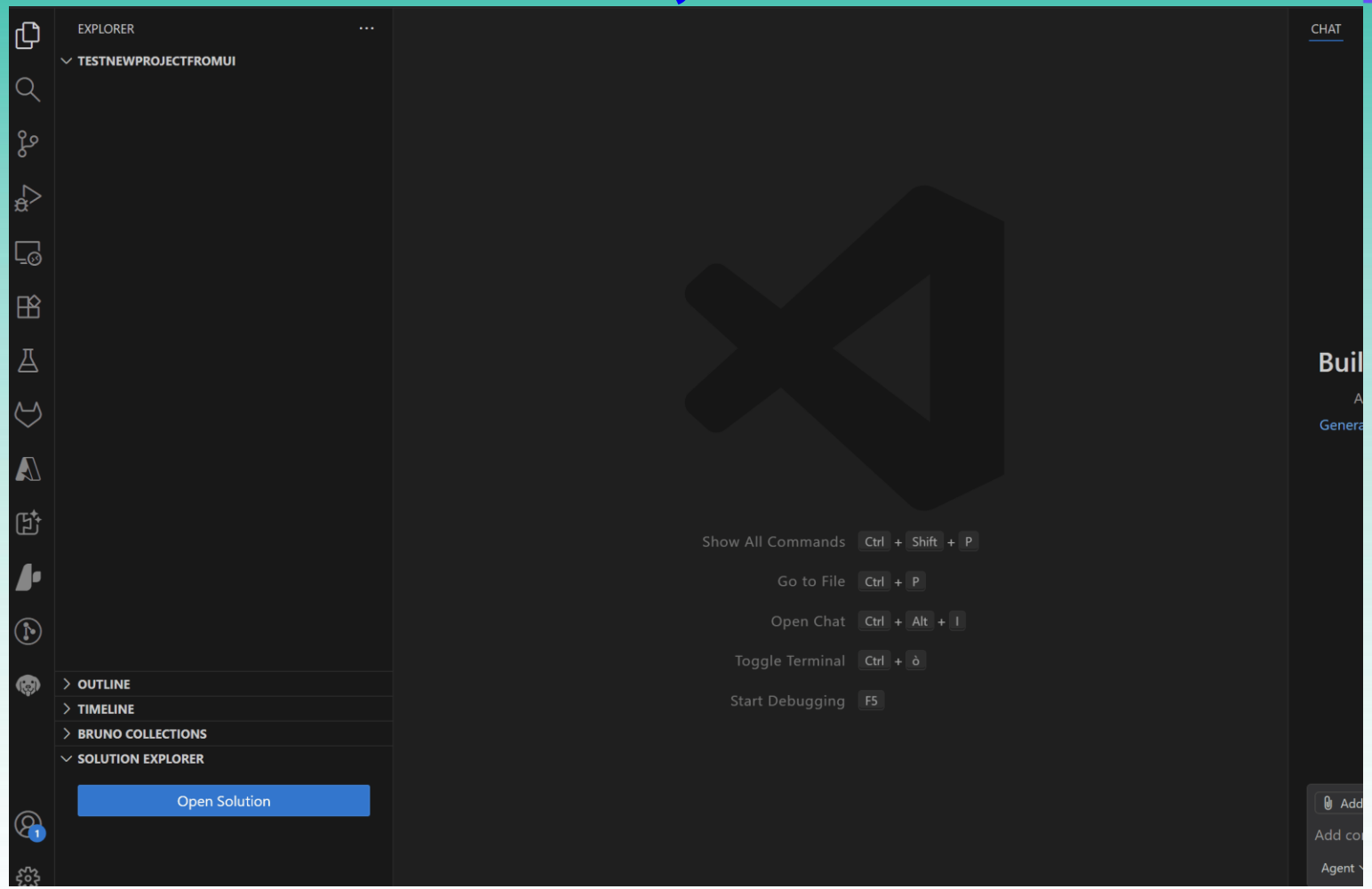
- Creare una nuova soluzione vuota
```
dotnet new sln -n NomeSoluzione
```

- Creazione dei progetti
```
dotnet new console -n ProgettoConsole
dotnet new webapi -n ProgettoWebAPI
```

- Aggiunta dei progetti alla soluzione
```
dotnet sln NomeSoluzione.sln add ./ProgettoConsole/ProgettoConsole.csproj
dotnet sln NomeSoluzione.sln add ./ProgettoWebAPI/ProgettoWebAPI.csproj
```

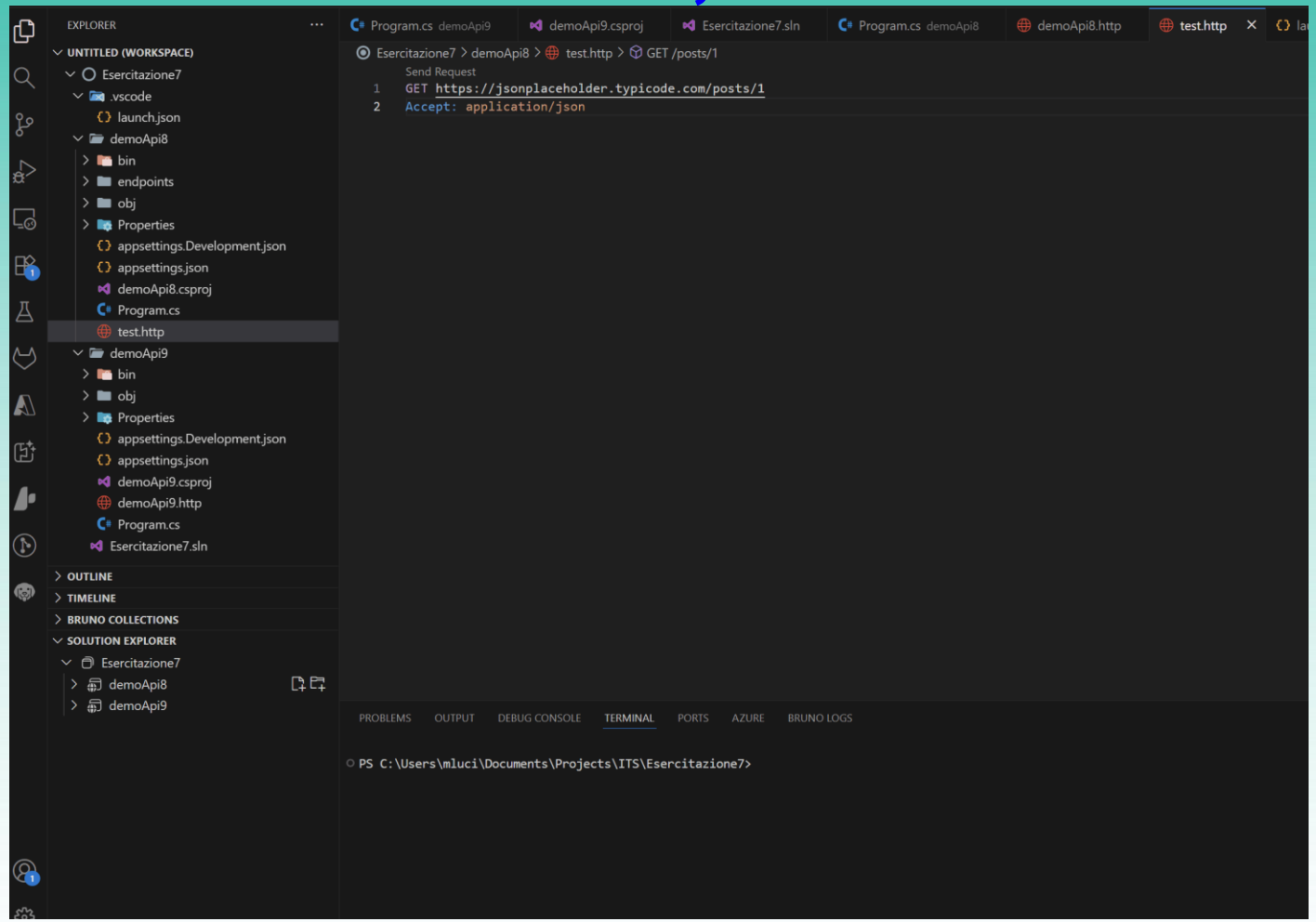
- Verifica dei progetti inclusi nella soluzione
```
dotnet sln NomeSoluzione.sln list
```

Addio terminale, benvenuta GUI 🙏



1. CTRL+SHIFT+P
2. > .NET New Project
3. Cercare il template progetto desiderato (es. webapi, console…)
4. Seguire i passi guidati
5. In «Show all template options» sarà possibile modificare la versione del framework, come anche altre impostazioni che possono tornare utili (es. debug…)

Addio terminale, benvenuta GUI 🙏



In ‘Solution Explorer’ si ha la possibilità di gestire la soluzione in modo estremamente semplificato come:

- Aggiungere o Rimuovere un nuovo progetto
- O aggiungerne uno già esistente
- Fare la build o il run dei progetti
- **Aggiungere i package da nuget!!!** (o anche rimuoverli)
- E altro ancora…

Esercitazione Finale - RacingSystem

Requisiti

🎯 Framework .NET 9	Tutti i progetti usano .NET 9
📦 Soluzione con 2 progetti	API + Console App
🌐 Web API	6 endpoint REST funzionanti
💻 Console Application	Menu interattivo completo
🗄️ **SQLite**	**Database con i dati persistenti**
🤖 **ML.NET**	**Predizione tempi**
🏗️ Pattern CSR	Controller-Service-Repository
🎓 Concetti OOP	Incapsulamento, Ereditarietà, Polimorfismo

Nelle prossime slide focus su ML.NET e SQLite (da utilizzare in quest'esercitazione finale)

ML.NET

Cos'è ML.NET?

Libreria Microsoft per fare **Machine Learning in C#** senza Python.

Come lo andremo ad usare?

Predice il tempo sul giro analizzando dati storici delle gare (potenza, peso, velocità → tempo previsto).

```
// Creiamo il dataset
var dataView = mlContext.Data.LoadFromEnumerable(trainingData);

// Pipeline: trasformiamo i dati e creiamo il modello
// Usiamo la regressione per predire il tempo sul giro
var pipeline = mlContext.Transforms.Concatenate("Features",
        new[] { "Power", "Weight", "TopSpeed" })
    .Append(mlContext.Regression.Trainers.Sdca(
        labelColumnName: "LapTime",
        maximumNumberOfIterations: 100));

// Addestriamo il modello
model = pipeline.Fit(dataView);

return "Modello addestrato con successo con " + trainingData.Count + " gare!";
```

SQLite

Un motore di database SQL serverless (senza server) e auto-contenuto.
L'intero database, con tabelle, indici e dati, è memorizzato in un unico file sul disco

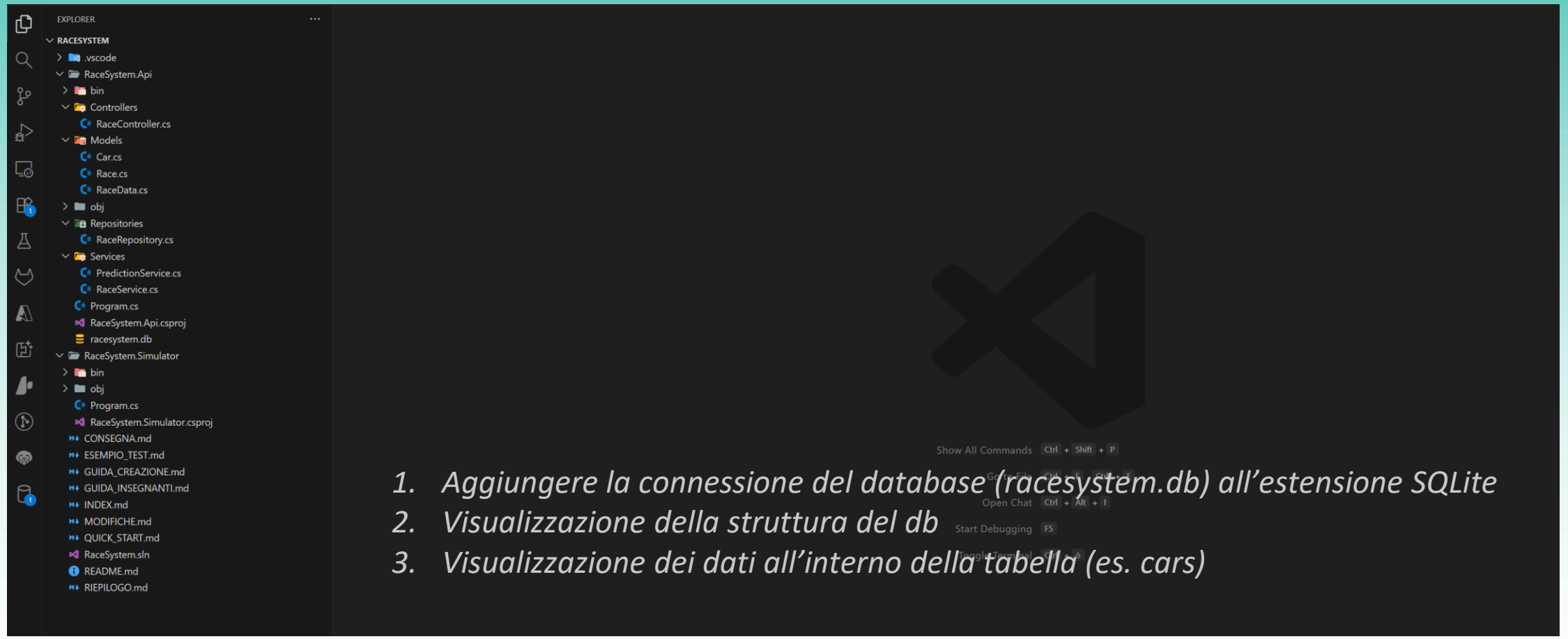


✅ Vantaggi
- Nessuna installazione richiesta
- Database in un singolo file (portatile)
- Ideale per app mobile e desktop (leggero)

❌ Svantaggi
- Concorrenza limitata (non adatto per connessioni multi utente
- Non per la rete, progettato per l'uso locale
- Funzionalità ridotte.

Estensioni VS Code da installare (entrambe) per SQLite:
https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite
https://marketplace.visualstudio.com/items?itemName=mtxr.sqltools-driver-sqlite

SQLite – Utilizzo dell'estensione per VS Code



1. *Aggiungere la connessione del database (racesystem.db) all'estensione SQLite*
2. *Visualizzazione della struttura del db*
3. *Visualizzazione dei dati all'interno della tabella (es. cars)*

SQLite – Pillole sulle istruzioni SQLite

RaceRepository.cs
La classe utilizza SQLite per gestire il database delle gare automobilistiche.

Connessione al database: `"Data Source=racesystem.db"`
Apertura connessione: `connection.Open()`
Utilizzo pattern `using` per chiusura automatica della connessione

Creazione ed esecuzione comandi:
Creare comando: `var command = connection.CreateCommand()`
Impostare query SQL: `command.CommandText = "SELECT * FROM Cars"`
Aggiungere parametri: `command.Parameters.AddWithValue("@id", id)`

Esecuzione query:
`ExecuteNonQuery()`: per INSERT, UPDATE, DELETE, CREATE TABLE
`ExecuteReader()`: per SELECT (restituisce SqliteDataReader)
`ExecuteScalar()`: per query che restituiscono un singolo valore (es. COUNT)

Lettura dati:
Iterare sui risultati: `while (reader.Read())`
Accedere ai valori per indice: `reader.GetInt32(0)`, `reader.GetString(1)`, `reader.GetDouble(3)`
Ogni metodo Get corrisponde al tipo di dato della colonna

Esercitazione Finale - RacingSystem

- Guida Step-by-Step (File .MD)

Per seguire tutti i passaggi di creazione del progetto fino alla fine

https://github.com/maurijph/EsercitazioneFinale/blob/88a5bcb3a571190b7e3e93c379250094b1320190/GUIDA_CREAZIONE.md

- Progetto Finale (Soluzione)

Il download del progetto già completato e funzionante (si spera)

https://github.com/maurijph/EsercitazioneFinale.git

- **Consiglio (SPASSIONATO!!!):**

Provate a completare l'esercitazione da soli prima di consultare la soluzione!

Considerazioni finali



Se anche voi vi sentite un po' così...

È del tutto normale!

Grazie per il percorso fatto insieme
e in bocca al lupo per tutto!